3F: Grunt

Security Review

Cantina Managed review by:
M4rio.eth, Lead Security Researcher
Chinmay Farkya, Security Researcher

May 13, 2026

Contents

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

From Mar 16th to Apr 13th the Cantina team conducted a review of grunt on commit hash 7056bb17, then the 3F team updated the commit hash to 878a5042. The team identified a total of **48** issues:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	1	1	0
Medium Risk	7	3	4
Low Risk	28	6	22
Gas Optimizations	0	0	0
Informational	12	7	5
Total	**48**	**17**	**31**

The Cantina Managed team reviewed 3F's grunt holistically on commit hash 809002b8 and concluded that all findings were addressed and no new vulnerabilities were identified.

2.1 Scope

The security review had the following components in scope for grunt on commit hash 7056bb17:

```
src
├── borrow
│   ├── MorphoBorrowPosition.sol
│   └── MorphoBorrowPositionFactory.sol
├── facility
│   ├── Facility.sol
│   ├── IntentDescriptor.sol
│   └── base
│       ├── FacilityFunds.sol
│       ├── FacilityIntents.sol
│       ├── FacilityLP.sol
│       ├── FacilityPositionManager.sol
│       ├── FacilityRequests.sol
│       ├── FacilityRoles.sol
│       └── FacilitySwap.sol
├── funds
│   ├── USCCFund.sol
│   ├── USCCFundFactory.sol
│   └── WrappedAsset.sol
├── guard
│   ├── TransferGuard.sol
│   └── TransferGuardFactory.sol
├── libs
│   ├── Constants.sol
│   ├── borrow
│   │   ├── LibBorrowErrors.sol
│   │   ├── MorphoBalancesLib.sol
│   │   └── SharesMathLib.sol
│   ├── common
│   │   ├── LibChecks.sol
│   │   ├── LibCommonErrors.sol
│   │   └── LibPause.sol
│   ├── facility
│   │   ├── LibAddress.sol
│   │   ├── LibConstants.sol
│   │   ├── LibFacilityErrors.sol
```

```
│   │   ├── LibIntent.sol
│   │   ├── LibStorage.sol
│   │   └── LibTokenBalances.sol
│   ├── funds
│   │   ├── LibFundsErrors.sol
│   │   └── Order.sol
│   ├── manager
│   │   ├── LibConstants.sol
│   │   ├── LibExecutor.sol
│   │   ├── LibManagerErrors.sol
│   │   ├── LibOperations.sol
│   │   ├── LibStorage.sol
│   │   └── LibView.sol
│   └── request
│       ├── Lib128Fields.sol
│       ├── LibAllowance.sol
│       ├── LibMintAuth.sol
│       ├── LibRequestErrors.sol
│       └── LibTokenController.sol
├── manager
│   ├── PositionManager.sol
│   ├── PositionManagerFactory.sol
│   ├── base
│   │   ├── PositionManagerAdmin.sol
│   │   ├── PositionManagerBase.sol
│   │   ├── PositionManagerLP.sol
│   │   └── PositionManagerRebalancing.sol
│   └── rebalancer
│       └── MorphoRebalancer.sol
└── request
    ├── Request.sol
    ├── RequestFactory.sol
    ├── Vault.sol
    └── abstract
        ├── OfferReceiver.sol
        ├── tokens
        │   ├── ControlledToken.sol
        │   └── TokenController.sol
        └── vault
            ├── ControlledVault.sol
            └── VaultController.sol
```

3 Findings

3.1 High Risk

3.1.1 Anyone can permanently delay `setRepaid()` by calling `mint(0, 0)`

Severity: High Risk

Context: *(No context files were provided by the reviewer)*

Description: `Request.mint()` at commit `2a5e885` takes two slippage parameters and reads the caller's mint authorization before checking them:

```
// Request.sol (commit 2a5e885)
function mint(uint128 minPt, uint128 minYt) external {
    if (_syncWithdrawalStatus()) revert LibRequestErrors.AlreadyRepaid();
    (uint128 ptMintAuth, uint128 ytMintAuth) = msg.sender.mintAuth();
    if (ptMintAuth < minPt || ytMintAuth < minYt) revert
    ↪  LibRequestErrors.SlippageExceeded();
    msg.sender.updateMintAuth(0, 0);
    _asset().safeTransferFrom(msg.sender, address(this), ptMintAuth);
    _mint(msg.sender, ptMintAuth, ytMintAuth);
    _requestStorage().lastMintTimestamp = uint40(block.timestamp);
}
```

An address with no mint authorization has `mintAuth() == (0, 0)`. Calling `mint(0, 0)` passes the slippage check (`0 < 0 || 0 < 0` is false), does a zero-value `safeTransferFrom` (accepted by standard ERC20s), mints zero PT and YT, and unconditionally updates `lastMintTimestamp` to `block.timestamp`.

`setRepaid()` enforces a `mintToRepaidDelay` since the last mint:

```
// Request.sol
uint40 _availableAt = _lastMint + req.mintToRepaidDelay;
if (block.timestamp < _availableAt) revert
↪  LibRequestErrors.SetRepaidTooEarly(_availableAt);
```

An attacker can call `mint(0, 0)` repeatedly, once per block, to keep `lastMintTimestamp` fresh and permanently block `setRepaid()` from succeeding. This stops PT and YT holders from ever withdrawing their principal and yield.

No authorization, funds, or special role is required. The cost is only gas, and the token must support zero-value transfers.

Recommendation: Consider adding:

```
if (ptMintAuth == 0 && ytMintAuth == 0) return;
```

This stops callers with no mint authorization from reaching the `lastMintTimestamp` update.

3F: Fixed in commit 69a9a320.

Cantina Managed: Fix verified.

3.2 Medium Risk

3.2.1 Facility can resolve and pay out pulled request principal after maturity while PT holders remain unpaid

Severity: Medium Risk

Context: *(No context files were provided by the reviewer)*

Description: `Facility.pull()` credits Request principal into the intent's tracked balances. After `repaymentDeadline`, `_syncWithdrawalStatus()` marks the request as repaid and `repay()` becomes unusable. `Facility.resolve()` calls `checkRequestRepaid()`, which calls `syncRepaidStatus()` on the Request (`LibIntent.sol:143-148`). Once the deadline passes, that call returns `true`, not because the loan was cleared, but because the deadline auto-flip fired. The intent resolves, LP claimants withdraw the pulled principal, and PT holders are left with an empty Request.

```
// Request.sol - _syncWithdrawalStatus()
if (block.timestamp >= req.repaymentDeadline) {
    req.repaid = true;
    emit Repaid(IERC20(_asset()).balanceOf(address(this)));
    return true;
}
```

```
// Request.sol - repay()
function repay(uint256 amount) external {
    if (_syncWithdrawalStatus()) revert LibRequestErrors.AlreadyRepaid();
    // ...
}
```

After `pull()` moves principal into the Facility (`FacilityRequests.sol:30-42`) and maturity passes, the normal repayment path via `Facility.repay()` reverts because `Request.repay()` is gated by the same sync (`Request.sol:331-332`). `Facility.resolve()` succeeds because `checkRequestRepaid()` sees the auto-flipped flag as a normal clearance (`FacilityIntents.sol:105-121`). The pulled principal, already sitting in intent balances, is then claimable by LP holders.

An attacker with facilitator access can combine this with an attacker-controlled LP to drain essentially all pulled principal from the Request side once maturity passes.

Recommendation: There are two ways to handle this:

- **Option 1 - Document and accept:** Document that maturity should always be set far enough in the future that pulled principal is returned before the deadline fires. If the maturity window is long enough, the auto-flip never races against outstanding pulls. This is an operational constraint on the facilitator when setting up requests.
- **Option 2 - Code fix:** Split the deadline into two distinct purposes. Right now it does two things at once: unlocking PT/YT withdrawals and satisfying `Facility.resolve()`. These should be separate.
- The deadline should keep firing PT/YT withdrawal unlock, which is its intended purpose as a last-resort guarantee for prime brokers.
- `Facility.resolve()` should require a separate explicit confirmation that the pulled principal has been returned, independent of whether the deadline has passed. The owner's `setRepaid()` call already provides this in the normal flow. Make `checkRequestRepaid()` require it unconditionally rather than accepting the deadline auto-flip as a substitute.

3F: Acknowledged. This is supposed to be set far in the future.

Cantina Managed: Acknowledged.

3.2.2 One blocked payout token bricks the entire `claim()` loop after LP burn succeeds

Severity: Medium Risk

Context: *(No context files were provided by the reviewer)*

Description: `claim()` in `FacilityLP.sol` burns LP shares first, then loops over every token in the intent's `amounts` map, calling `transferTokenTo()` for each positive payout. A revert on any single token aborts the entire claim after the burn-side checks have already passed.

```
// FacilityLP.sol:92-101
for (uint256 i = 0; i < length; i++) {
```

```
    address token = tokens[i];
    uint256 userBalance = _intent.amounts.get(token).mulDiv(shares, supply);
    amounts[i] = userBalance;
    if (userBalance > 0) {
        _intent.transferTokenTo(id, token, receiver, userBalance);  // reverts whole tx
        ↪  if blocked
    }
}
```

`src/facility/base/FacilityLP.sol`

The PositionManager `TransferGuard` is a concrete trigger: PM shares carry per-address transfer restrictions. An LP holder may be allowed to burn their LP shares (the burn check passes), yet the `receiver` they specify for the payout may be blocked from receiving PM shares, causing the PM-share leg to revert and taking the entire claim down with it. Residual debt, collateral dust, or recovery tokens that remain in `amounts` after resolve can also block the loop if those tokens have any transfer restriction.

Extra tokens can appear in various edge cases:

- Borrowing into an intent with no Request leaves residual debt in `amounts`.
- Partial Request repayment leaves residual debt in `amounts`.
- Partial PM deposit, slippage, or dust leaves residual collateral in `amounts`.
- Recovery/retry flows can also resolve with residual non-PM assets tracked.

Recommendation: Split claim into two steps: first, an allocation step that loops over every token and assigns amounts to the user; second, a permissionless `claimToken` function that claims one assigned token for a given user. The user can then pick which tokens to claim, so one blocked token doesn't prevent the rest.

3F: Acknowledged. But probably problematic with wUSCC that will check the superstate allowlist. With Centrifuge for example, they are able to receive the assets. We don't know if it will happen, but we can push back for now.

Cantina Managed: Acknowledged.

3.2.3 `virtualShareOffset = 1` on 18-decimal debt PMs provides no inflation protection, donation attack steals victim deposits

Severity: Medium Risk

Context: *(No context files were provided by the reviewer)*

Description: $\text{virtualShareOffset} = 10^{(18 - \text{debtDecimals})}$ is intended to prevent donation-based share inflation. For 6-decimal tokens (USDC) the offset is 10^{12}, providing strong rounding protection. For 18-decimal tokens (DAI, WETH) the offset is `1`, which provides no protection.

```
// PositionManager.sol:68
// offset = 1 is "safe" for tokens with >= 18 decimals
virtualShareOffset = 10 ** (18 - debtDecimals); // = 1 when debtDecimals == 18
```

The comment claims offset = 1 is safe for 18-decimal tokens. It is not.

Attack path:

1. The attacker seeds the PM with 1 share via a normal `depositManager()` call.
2. The attacker calls Morpho's permissionless `supplyCollateral(onBehalf = sleeve)` to donate collateral directly to the PM's Morpho sleeve. This inflates `totalAssets()` without minting any PM shares.
3. Next victim deposits routed through `depositManager()` mint zero shares due to rounding:

   [] shares = assets * (totalSupply + offset) / (totalAssets + offset) = assets * (1 + 1) / (~donation + 1) 0 for any deposit < donation / 2

4. The attacker burns their 1 share. The burn formula gives:

   [] collateral = totalCollateral * shares / (totalSupply + offset) = totalCollateral * 1 / (1 + 1) = ~50% of the entire pool

5. Victims receive nothing. The attacker nets approximately 50% of all victim deposits minus the donation cost.

The attack requires no special role. `morpho.supplyCollateral(onBehalf = ...)` is permissionless. Guardian signatures are bypassed entirely since the attacker only calls `depositManager()` once to seed the PM, then donates externally.

Note: The issue has been marked as Medium because of various constraints on this attack, including a compromised facilitator and a fresh PM with 1e18.

Recommendation:

- The `10^(18-decimals)` formula works well for low-decimal tokens (e.g. USDC gets `10^12`). The problem is only 18-decimal tokens where the offset collapses to 1. Set a minimum floor on the offset, e.g. `max(10^(18-decimals), 10^6)`, so it never drops below meaningful protection while keeping the existing scaling for low-decimal markets.
- The offset must also flow through the burn-side math not just the mint side.
- A protocol seed deposit on PM setup adds defense-in-depth but is not a standalone fix.

3F: Fixed in PR 167.

Cantina Managed: PR 167 reduces attacker profit but does not fully eliminate the attack. For 18-decimal tokens, 3F will ensure a first deposit is made to bootstrap the `PositionManager`. The remaining risk requires operational mitigations, as the scenario also assumes a compromised facilitator.

3.2.4 PT and YT ERC-4626 view functions return inconsistent quotes depending on request state and live balance

Severity: Medium Risk

Context: *(No context files were provided by the reviewer)*

Description: PT and YT implement ERC-4626 views, but the numbers they return are wrong or misleading in a few cases.

1. Live balance reporting creates a misprice window during pulls.

All pricing is derived from the live on-chain balance of the Request contract:

```
// VaultController.sol:68-79
uint256 assets = _asset().balanceOf(address(this));
pAssets = min(assets, ptSupply);
yAssets = assets - pAssets;
```

`Facility.pull()` transfers assets out of the Request before the prime broker repays them. While those funds are out, every call to `totalAssets`, `convertToAssets`, `previewRedeem`, and `previewWithdraw` on the PT and YT vaults reflects the reduced balance. Any 3rd party integrator reading these functions during a normal `Facility.pull()` call sees a deflated value.

2. `convertToAssets` flips to par when the balance reaches zero.

`convertToAssets` special-cases the zero-balance pre-repayment state and returns a 1:1 estimate instead of the live balance quote:

```
// VaultController.sol:223-236
pAssets = (totalPtSupply == 0 || (totalPAssets == 0 && !repaid))
    ? _initialConvertToAssets(ptShares, false)   // 1:1 estimate
    : ptShares.mulDiv(totalPAssets, totalPtSupply); // live balance quote
```

Pulling all but one unit of backing drops the quote, but pulling the last unit makes it jump back to par:

State	repaid	assets	ptSupply	PT `convertToAssets(100)`
Partially drained	false	70	100	70
Fully drained	false	0	100	100

This same branch can also be triggered by a 1-wei direct token transfer to the Request address. Before repayment, when `balance == 0`, the function takes the initial 1:1 path. A 1-wei donation switches it to the `mulDiv` path, which rounds to 0 for typical share amounts (e.g., `ptShares.mulDiv(1, 1000e6)` returns 0). The cost to an attacker is 1 wei. The Request's own `maxRedeem` and `canWithdraw` correctly block redemptions pre-repayment, so no fund loss occurs, but any 3rd party integrator reading `convertToAssets()` or `previewRedeem()` without first checking `maxRedeem > 0` gets a wrong value.

3. `previewWithdraw` and `maxWithdraw` / `maxRedeem` are unreliable.

`previewWithdraw` delegates to `convertToShares`, which inherits the same zero-balance fallback. A fully drained vault can return a nonzero `previewWithdraw` quote while the actual `withdraw` call reverts on the token transfer, misleading integrators that simulate before executing.

`maxWithdraw` and `maxRedeem` gate on the stored `canWithdraw()` flag, which is only updated by `_syncWithdrawalStatus()` on state-changing paths. Around maturity, `redeem` and `withdraw` succeed because they call `_syncWithdrawalStatus()` internally, but `maxWithdraw` and `maxRedeem` still report `0` until a separate transaction updates the stored flag. Integrators checking `max*` before calling `redeem` will think no exit is available when there actually is one.

4. First post-deadline `redeem()` can revert because sync runs after pricing.

`_redeem()` calls `convertToAssets()` before `_withdrawalOperation()`. Since `convertToAssets()` reads the stored `repaid` flag (not the deadline), the first `redeem()` after deadline passes but before anyone syncs uses stale pricing. In the zero-asset case this is a real revert: `convertToAssets()` sees `totalPAssets == 0 && !repaid`, returns 1:1 (full principal), then `_withdrawalOperation()` syncs `repaid = true`, burns shares, and tries to transfer the full principal amount from an empty balance. The transfer reverts.

If `repaid` were already synced, `convertToAssets()` would take the `mulDiv(0, ptSupply) = 0` path, transfer 0, and succeed. So the first post-deadline redeem is broken until someone calls `syncRepaidStatus()` in a separate transaction.

`_withdraw()` is not affected today because `convertToShares()` does not branch on `repaid`. But the ordering is still wrong for consistency.

Recommendation: Two things to fix:

View functions (`maxWithdraw`, `maxRedeem`, `convertToAssets`): Check `block.timestamp >= repaymentDeadline` directly instead of reading the stored `repaid` flag. This way these functions reflect the actual state without waiting for a sync transaction.

`_redeem()` and `_withdraw()` ordering: Move `_syncWithdrawalStatus()` before the conversion call. Right now `_redeem()` calls `convertToAssets()` first, then `_withdrawalOperation()` which syncs. Flip this so the repaid flag is current before pricing runs. Do the same for `_withdraw()` for consistency, even though `convertToShares()` doesn't branch on `repaid` today.

3F: Fixed in PR 185.

Cantina Managed: PR 185 is a partial fix: it makes `maxWithdraw`, `maxRedeem`, and `canWithdraw` real-time by having `Request._canWithdraw()` return `repaid || block.timestamp >= repaymentDeadline`, which also ensures `convertToAssets()` observes the correct state on the first post-deadline call. Sub-issue 1 (live `balanceOf` misprice during `Facility.pull()`) was not fixed; `_assetsAndSupplies()` still reads `_asset().balanceOf(address(this))`.

3.2.5 `onRequestConsumed` callback can reenter unprotected Request functions

Severity: Medium Risk

Context: *(No context files were provided by the reviewer)*

Description: `Request.consume()` carries a `nonReentrant` guard, but the guard is not shared with other state-mutating entrypoints on the same contract. The optional maker callback fires before the principal transfer and the mint, so a maker contract can use `onRequestConsumed` to call back into `pullFunds()`, `mint()`, `authorizeMinting()`, or `setRepaid()` while `consume()` is still in progress.

```
// Request.sol:411-427 (simplified)
function consume(Offer calldata offer, ...) external nonReentrant returns (uint256
↪  ytAmount) {
    _syncWithdrawalStatus(); // repayment check, done once
    // ...
    if (offer.useCallback) {
        IRequestCallback(offer.maker).onRequestConsumed(offer, signature, ptAmount,
        ↪  ytAmount);
        // ← all other Request mutators are reachable here
    }
    _asset().safeTransferFrom(offer.maker, address(this), ptAmount); // principal pulled
    ↪  after callback
    _mint(offer.maker, ptAmount, ytAmount);
}
```

Attack 1: self-funded PT/YT issuance via pullFunds.

A maker contract that also holds `PULLER_ROLE` can use the callback to call `pullFunds()`, take assets out of the Request, approve the Request for the same amount, and return. `consume()` then pulls that same principal back in and mints PT/YT against it. The Request balance is restored, but PT/YT supply has increased without any net new capital.

Attack 2: repayment latched before mint.

A maker contract that also owns the Request can call `setRepaid()` from inside the callback. `consume()` checks `_syncWithdrawalStatus()` only once before the callback, so after the callback returns the Request is already in the repaid state while `consume()` continues minting fresh PT/YT. The freshly minted junior supply can redeem immediately against the pre-existing yield bucket, taking most of it while `repaidAvailableAt()` still points a full timelock period into the future.

Recommendation: Treat the whole Request as one reentrancy domain. Apply the same guard to `pullFunds()`, `mint()`, `authorizeMinting()`, and `setRepaid()` so none of them can be called while a `consume()` callback is in progress.

3F: Fixed in PR 177.

Cantina Managed: Fix verified.

3.2.6 Zero-clipped NAV in `LibView.totalAssets()` hides bad debt and can make PM users lose value

Severity: Medium Risk

Context: *(No context files were provided by the reviewer)*

Description: Once one sleeve goes underwater, PM accounting treats that sleeve as worth zero instead of negative, and everything that reads `totalAssets()` uses that inflated number.

`LibView.totalAssets()` computes PM value by summing each borrow sleeve's net equity with a zero floor:

```
function totalAssets(PositionManagerStorageData storage ps) internal view returns
↪  (uint256 amount) {
    address[] memory modules = ps.borrowModules.values();
    uint256 modulesLength = modules.length;
    for (uint256 i = 0; i < modulesLength; ++i) {
        uint256 collateral = IBorrowPosition(modules[i]).totalCollateralQuoted();
        uint256 debt = IBorrowPosition(modules[i]).totalBorrowed();
        amount += collateral.zeroFloorSub(debt);
    }
}
```

`zeroFloorSub` caps a sleeve's contribution at zero once debt exceeds collateral. This is intentional: one bad sleeve should not drag the whole PM NAV below zero. The problem is that this clipped number is then used as source of truth for share pricing in `_settleShares()`, fee accrual, rebalance loss checks, and Facility intent resolution. Not every PM flow reads it (`burn()` pulls raw sleeve totals directly), but all the ones that do have the same problem.

Once any sleeve goes underwater, `totalAssets()` stops tracking what the PM actually holds. The sleeve stays active: interest keeps accruing, public liquidations can happen, and oracles keep moving. None of that shows up in the clipped PM number. Users priced against this number get a value that doesn't match reality.

Path 1: New deposits subsidize existing holders: While a sleeve is underwater and accounting is clipped, any new deposit (direct, or routed through the Facility via `depositManager`) mints shares using an inflated `totalAssets()`. The new depositor fills the hole for existing holders at a bad price: they pay full value for shares that represent a smaller slice of the true NAV, and the hidden loss spreads across the now-larger shareholder set.

Example: true net NAV is 40 but `totalAssets()` reports 60. A new depositor putting in 10 gets `10/60` of the shares. They actually bought into a pool worth 40, so the fair share would have been `10/40`. The difference moves value from the new depositor to existing holders.

This also applies to the Facility PM helpers (`FacilityPositionManager.depositManager`, `withdrawManager`, `burnManager`). Intents that resolved before the sleeve went bad hold Facility-owned PM shares, and later clipped-NAV operations reprice those already-resolved intents at the wrong share price.

Path 2: A PM shareholder can pocket Morpho's liquidation bonus from an already-bad sleeve: Once a sleeve is underwater, anyone can still call Morpho's native `liquidate()` on it. Morpho's native liquidation pays a Liquidation Incentive Factor (LIF), usually around 15%, to whoever calls it. `preLiquidate()` does not pay that bonus.

Setup:

- PM has borrow position A (underwater): 80 collateral value, 90 debt, so `zeroFloorSub` contribution = **0**.
- PM has borrow position B (healthy): 100 collateral value, 50 debt, so contribution = **50**.
- PM `totalAssets()` = 0 + 50 = **50**.
- Attacker separately holds some PM shares.

Attack:

Attacker calls Morpho's native `liquidate()` on position A (not `preLiquidate`):

- Repays **20** of position A's debt.
- Seizes **~23** of position A's collateral (20 × 1.15 LIF bonus).
- Attacker's profit outside the PM: `23 - 20 = +3`.

After the liquidation:

- Position A: 57 collateral value, 70 debt, still underwater, so `zeroFloorSub` contribution = **0**.

- Position B: unchanged, contribution = **50**.
- PM `totalAssets()` = 0 + 50 = **50** (unchanged).

The PM lost 23 of real collateral from sleeve A and only got 20 of debt reduction back. That net 3-unit hit is hidden by the zero floor and never shows up in `totalAssets()`. The attacker keeps the LIF bonus while everyone holding PM shares takes the loss. Net external profit for the attacker is roughly `3 × (1 - attacker_share_ownership)`.

Path 3: `withdraw()` and `burn()` stop agreeing once a sleeve is bad: `withdraw()` and `burn()` are the two ways to exit a PM position. Once the PM has a clipped sleeve, they use different math and the user who picks the cheaper path takes more than their fair share.

`withdraw(collateral, debt, strategy)` captures `totalAssetsBefore = _accrueFees()` (clipped), processes the withdrawal through the queue, then calls `_settleShares(totalAssetsBefore, _totalSupply)`. That helper reads `totalAssets()` again (still clipped) and burns shares based on the clipped delta:

```
// PositionManagerLP.sol:183
function _settleShares(uint256 totalAssetsBefore, uint256 _totalSupply) internal returns
↪  (int256 sharesDelta) {
    uint256 totalAssetsAfter = _storage.totalAssets();        // clipped
//...
    } else if (totalAssetsAfter < totalAssetsBefore) {
        uint256 assetsRemoved = totalAssetsBefore - totalAssetsAfter;
        uint256 sharesToBurn = assetsRemoved.convertToShares(_totalSupply,
        ↪  totalAssetsBefore, virtualShareOffset_, true);
        _burn(msg.sender, sharesToBurn);
//...
    }
}
```

`burn(shares, strategy)` never calls `_settleShares()` and never reads `totalAssets()`. It pulls raw collateral and debt totals straight out of storage and gives the caller their proportional slice:

```
// PositionManagerLP.sol:134-144
uint256 _totalCollateral = _storage.collateralAmount();   // raw sum, no clipping
uint256 _totalDebt = _storage.debtAmount();               // raw sum, no clipping
//...
collateral = _totalCollateral.mulDiv(shares, _totalSupply + virtualShareOffset_);
debt = _totalDebt.mulDivUp(shares, _totalSupply + virtualShareOffset_);

_burn(msg.sender, shares);   // burns exactly the shares the caller asked for
```

`collateralAmount()` and `debtAmount()` in `LibView.sol` are simple unclipped sums. In the bad-sleeve window, `_settleShares()` charges share cost based on a clipped delta that is smaller than the real delta. A user who gets the same collateral and debt via `withdraw()` burns fewer shares than if they used `burn()`, and the remaining shareholders absorb the difference.

Path 4: Fee shares can be minted on gains that did not happen: The clipped NAV breaks fee accrual. `_pendingFees()` compares current `totalAssets()` against the stored `lastTotalAssets` snapshot to check for performance gain:

```
if (fd.performanceFee > 0 && totalAssets_ > _lastTotalAssets + managementFeeAssets) {
    uint256 gains = totalAssets_ - _lastTotalAssets - managementFeeAssets;
    uint256 performanceFeeAssets = gains.mulDiv(fd.performanceFee, BPS);
//...
}
```

Both sides are clipped. Example:

- Last clean snapshot: `lastTotalAssets = 50`.
- Current portfolio: one sleeve at +60 NAV, one sleeve at -20 NAV.

- **Actual portfolio value:** 60 - 20 = **40** (a real loss of 10).
- **What `totalAssets()` reports:** 60 + max(-20, 0) = **60** (a phantom gain of 10).

The next fee accrual thinks the manager went from 50 to 60 and mints performance fee shares on that fake 10 of "profit," even though the portfolio went from 50 to 40. Then `_accrueFees()` saves `lastTotalAssets = 60`, which skews the baseline going forward: if the bad sleeve later recovers and real NAV climbs from 40 back to 60, the baseline is already at 60 and that real recovery triggers no new performance fee.

Path 5: Cleanup rebalances can look like new loss even when nothing changed: `PositionManagerRebalancing.rebalance()` measures loss from the clipped PM NAV:

```
if (totalAssetsAfter < totalAssetsBefore) {
    uint256 loss = totalAssetsBefore - totalAssetsAfter;
    if (loss * BPS > uint256(_storage.rebalanceConfig.maxRebalanceLoss) *
    ↪  totalAssetsBefore) {
        revert LibManagerErrors.RebalanceLossExceedsMax();
    }
}
```

Both `totalAssetsBefore` and `totalAssetsAfter` come from the clipped `totalAssets()`. Any rebalance that actually repays the bad sleeve's debt can "unhide" loss that the clamp was hiding. Example:

- Before: sleeve A = 80 collateral / 90 debt (clipped to 0), sleeve B = 100 collateral / 50 debt (contribution 50). `totalAssets = 50`.
- Rebalancer provides 20 debt asset, uses a REPAY operation to clear 20 of sleeve A's debt, and uses a WITHDRAW operation to pull 20 collateral out of sleeve B. Net for the rebalancer: they swapped 20 debt for 20 collateral. PM's externally-visible value is unchanged.
- After: sleeve A = 80 / 70 (+10 contribution), sleeve B = 80 / 50 (+30 contribution). `totalAssets = 40`.

True NAV is unchanged, but the clipped NAV drops from 50 to 40 because the hidden 10 of bad debt from sleeve A is now visible after the cleanup. The loss check sees a 10-unit drop and can trip. Any `maxRebalanceLoss` tight enough to catch real slippage is likely too tight to tolerate a chunk of previously-hidden bad debt suddenly appearing.

Recommendation: There is no clean in-contract fix. The zero floor in `LibView.totalAssets()` is intentional so one bad sleeve cannot drag the whole PM NAV below zero. Carrying signed per-sleeve NAV through share pricing, fee accrual, and rebalance loss accounting comes with its own issues.

The real fix is operational. As soon as a sleeve goes bad, pause the Facility, stop resolving new intents, pull the damaged module out of the PM routing queues, and remove the broken sleeve from `borrowModules` so it stops affecting accounting. Every path above is bounded by how fast the team notices and reacts, so user safety in this window depends on monitoring and how quickly the team can run the abandonment playbook.

3F: Acknowledged.

Cantina Managed: Acknowledged.

3.2.7 `wUSCC`-backed Morpho flows require Superstate-allowlisted protocol recipients and liquidators

Severity: Medium Risk

Context: *(No context files were provided by the reviewer)*

Description: `wUSCC` transfers have to pass two separate checks in `WrappedAsset._beforeTokenTransfer(...)`:

```
if (from != address(0) && to != address(0) && !hasAnyRole(to, RECEIVER_ROLE) &&
↪  !hasAnyRole(from, SENDER_ROLE)) {
    revert Unauthorized();
```

```
}
if (from != address(0) && !isAllowed(from, amount)) revert Unauthorized();
if (to != address(0) && !isAllowed(to, amount)) revert Unauthorized();
```

Granting `SENDER_ROLE` / `RECEIVER_ROLE` is not enough by itself. Every non-null sender and receiver must also pass `isAllowed(...)`. On `wUSCC`, that second check is not local policy; `SuperstateRestrictedWrappedAsset.isAllowed(...)` delegates straight to `USCC.isAllowed(account)`.

So every step in the Morpho collateral flow needs Superstate allowlisting:

- `PositionManager.deposit(...)` first pulls `wUSCC` from the caller into the PM (`PositionManagerLP.sol#L54-L57`), so the `PositionManager` itself must be Superstate-allowlisted as a recipient.
- The same deposit path then forwards collateral from the PM into a `MorphoBorrowPosition` through `LibOperations.processDeposit(...)` and `MorphoBorrowPosition.supplyCollateral(...)`, so each borrow-position sleeve must also be Superstate-allowlisted.
- Morpho then pulls the same `wUSCC` into the market contract in `Morpho.supplyCollateral(...)`, so the Morpho core contract must be Superstate-allowlisted too.

We assume that these will be whitelisted by Superstate, but the dependency is even more serious on liquidation. Both the custom pre-liquidation path and native Morpho liquidation send `wUSCC` directly to the liquidator:

- `MorphoBorrowPosition.onMorphoRepay(...)` calls `MORPHO.withdrawCollateral(..., liquidator)`;
- `Morpho.withdrawCollateral(...)` finally does `safeTransfer(receiver, assets)`.

With `wUSCC` as collateral, the liquidator must also satisfy `USCC.isAllowed(liquidator)`. The market is no longer openly liquidatable. A position can only be liquidated by the subset of actors that Superstate has allowlisted to receive `USCC` / `wUSCC`.

This is how restricted assets work. Before the new transfer guard, any liquidator could repay debt and receive the wrapped asset, then get whitelisted separately to unwrap it. Now, liquidators must be Superstate-whitelisted just to receive the collateral during liquidation.

Recommendation: Document this risk clearly. Before opening wUSCC-backed markets, onboard a list of Superstate-allowlisted liquidators so there are enough players ready to liquidate if bad debt starts building up.

3F: Acknowledged.

Cantina Managed: Acknowledged. Previously, a liquidator could be any address that repaid debt (and would only face the whitelist check at unwrap time). With the new transfer-guard readaptation, the Superstate allowlist is checked preemptively on collateral receipt, so liquidators must now be whitelisted by Superstate to receive `wUSCC` during liquidation. The risk model has changed.

3.3 Low Risk

3.3.1 Raw `USCCFund` balances credited to wrong order, wrong intent, and wrong LP group

Severity: Low Risk

Context: *(No context files were provided by the reviewer)*

Description: `USCCFund` does not track value at the order level. `_state()` decides whether the current order can unlock or recover by comparing the raw `balanceOf(address(this))` against the order's expected output or input:

```
// Deposit: check if we received USCC
_amount = IERC20(USCC).balanceOf(address(this));
return _amount >= _effectiveOutput ? (State.UNLOCKING, _amount) : (State.PROCESSING, 0);
```

When `_state()` returns `UNLOCKING`, `unlock()` transfers the full observed balance, not just the amount belonging to the current order. `Facility.unlock()` and `Facility.recover()` then snapshot the full Facility-side balance delta into the current intent through `commitBalanceSnapshot()`:

```
uint256 currentBalance = IERC20(snapshot.token).balanceOf(address(this));
if (currentBalance > snapshot.balance) {
    uint256 amount = currentBalance - snapshot.balance;
    _receivedTokenFrom(_self, id, snapshot.token, counterparty, amount);
}
```

Any balance sitting on the fund that isn't from the current order gets swept in: late USCC from an older order, leftovers from a previous flow, or tokens on a reused fund.

Example: an old deposit order ends, the same fund is reused for a new intent, and late USDC from the old flow lands on the fund. The new intent calls `unlock()`, sees the stale balance as part of its own order completing, and pays it out to the current LPs even though that value belonged to a different order.

The same bug is on the `recover()` path: a deposit order in `RECOVERING` picks up foreign USDC and pays it out as a refund to the current LPs through the Facility-side recover flow.

Recommendation: There are two ways to handle this:

- **Option 1 - Document and accept:** Leave as is, document that fund reuse across orders can cause late-arriving tokens to be swept into the wrong order, and account for this when creating orders.
- **Option 2 - Code fix:** Snapshot the fund's token balance at `commit()` time and use the before/after delta in `_state()`, `unlock()`, and `recover()` rather than the raw contract balance. Only tokens that arrived after the current order's commit are credited to it. Late arrivals from old orders land as unassigned in the contract rather than being swept into the current order. Assign these via the `OPERATOR` role, since the operator is trusted.

3F: Acknowledged. We don't want to add an operator assignment flow. That would introduce manual reconciliation and extra operational burden. Our preferred fix is to make the fund's behavior explicit: USCCFund is a rolling adapter with one active order at a time, and unlock() / recover() sweep the relevant token balance into that active order. In that model, late or residual balances are intentionally carried forward rather than left unassigned. If strict per-order attribution is required, balance snapshots alone are not enough.

Cantina Managed: Acknowledged.

3.3.2 Live PT supply accounting can divert later repayments to YT before PT is made whole

Severity: Low Risk

Context: *(No context files were provided by the reviewer)*

Description: `VaultController._assetsAndSupplies()` computes `pAssets = min(balance, ptSupply)` using the live PT supply at call time (`VaultController.sol:77`):

```
// VaultController.sol - _assetsAndSupplies()
(ptSupply, ytSupply) = LibTokenController.totalSupplies();
uint256 assets = _asset().balanceOf(address(this));
pAssets = FixedPointMathLib.min(assets, ptSupply);
yAssets = assets - pAssets;
```

If PT holders redeem while the request is underfunded, PT supply drops even though principal was never fully repaid. Later repayment inflows are then split using the reduced `ptSupply` as the ceiling, so assets that should cover the principal shortfall can instead be classified as `yAssets` and go to YT holders. LP

withdrawers who exit before a later repayment above the outstanding principal may also lose a portion of yield to other participants.

Example:

- 100 PT and 100 YT are issued; principal is pulled out.
- Withdrawals unlock with only 80 assets in the Request.
- A PT holder redeems 50 PT and receives 40 assets (pro-rata of the 80 available).
- PT supply is now 50, but the total principal obligation was 100 and only 80 ever arrived.
- 30 more assets are later sent directly to the Request.
- The Request now holds 70 assets with a live PT supply of 50.
- `pAssets = min(70, 50) = 50`, `yAssets = 20`.
- YT receives 20, even though the original 100-unit principal obligation was never fully honored.

Recommendation: Document that repayments made after partial PT redemptions can shift asset classification between PT and YT claimants.

3F: Acknowledged.

Cantina Managed: Acknowledged.

3.3.3 Facility pause and TransferGuard blocklist both block compliance-only `revertDeposit()`

Severity: Low Risk

Context: *(No context files were provided by the reviewer)*

Description: `revertDeposit()` is gated to `owner` and `COMPLIANCE_ROLE` and is intended to force-return a depositor's funds before resolution. Unlike `withdraw()` and `claim()`, it has no explicit `LibStorage.checkNotPaused()` call. However, because it burns LP shares via `_burn()`, it picks up the pause check through `Facility._beforeTokenTransfer()`, which calls `LibStorage.checkNotPaused()` on every mint, transfer, and burn.

1. `revertDeposit()` is blocked when the Facility is paused.

When the Facility is paused, `_burn()` -> `_beforeTokenTransfer()` -> `LibStorage.checkNotPaused()` reverts.

As a side effect, `withdraw()` and `claim()` each call `checkNotPaused()` explicitly in `_withdrawalLpChecks()` and then hit the same check again through `_burn()` -> `_beforeTokenTransfer()`. The double-check is harmless but suggests the team may not have accounted for the burn-side inheritance.

2. Blocklisting `from` permanently prevents `revertDeposit()` from executing.

`_beforeTokenTransfer()` also enforces the intent's live `TransferGuard`:

```
if (guard != address(0)) {
    if (!ITransferGuard(guard).canTransfer(_intent.properties.guardKey, from, to,
    ↪  amount)) {
        revert LibFacilityErrors.TransferBlocked(guard, from, to, amount);
    }
}
```

`TransferGuard.canTransfer()` checks the `from` address for burns (`to == address(0)`) the same as for transfers. Only the `to` check is skipped for burns:

```
// TransferGuard.sol
if (from != address(0) && !_isAllowed($, from, config.whitelist)) {
    return false;
}
```

```
// Check recipient (skip for burns)
if (to != address(0) && !_isAllowed($, to, config.whitelist)) {
    return false;
}
```

The compliance flow breaks itself:

1. A user is flagged as a bad actor.
2. Compliance blocklists the user on the `TransferGuard`.
3. Compliance calls `revertDeposit(id, from, receiver)` to return their deposit.
4. `_burn(from, id, balance)` -> `_beforeTokenTransfer(from, address(0), ...)` -> `canTransfer(guardKey, from, address(0), ...)` -> `from` is blocklisted -> reverts.

The blocklisted user's funds are permanently stuck. The only workaround is to un-blocklist the address, run `revertDeposit`, and re-blocklist, which means briefly un-blocking the bad actor just to return their money.

Recommendation: There are two ways to handle this:

- **Option 1 - Document and accept:** Document that `revertDeposit()` does not work on blocklisted addresses. Compliance must un-blocklist, revert the deposit, then re-blocklist. Accept the brief window where the bad actor is unblocked.
- **Option 2 - Code fix:** `revertDeposit()` is a privileged compliance path, not a user-initiated transfer. Add an internal burn variant that bypasses `_beforeTokenTransfer()` for use in `revertDeposit()`, so compliance can return funds to blocklisted addresses without un-blocking them first.

3F: Acknowledged. We will just `revertDeposit` and then block the address (could be atomic).

Cantina Managed: Acknowledged.

3.3.4 Already-repaid or expired request can be rebound to a new intent, bypassing the resolve

Severity: Low Risk

Context: *(No context files were provided by the reviewer)*

Description: `setRequest()` has no check on whether the incoming request is still live. It only checks that the *old* request is repaid before releasing it. The new request just needs to be a contract, not already mapped, and asset-compatible.

The repayment check on the old request is at `FacilityIntents.sol:211`. Once it passes, the old request is removed from the global `requestsIntent` map via `abandonRequest()` at `FacilityIntents.sol:236` (which calls `LibStorage.sol:189-190`). The uniqueness check on the new request at `FacilityIntents.sol:228` only verifies the request is not already bound to a different intent. It does not check whether the request is still active.

```
// FacilityIntents.sol - setRequest()
_intent.checkRequestRepaid();        // checks OLD request, not newRequest
// ...
newRequest.checkContract();
_facilityStorage.checkRequestIntent(newRequest, id);  // uniqueness only
(, address _pmDebt) = IPositionManager(_intent.properties.guardKey).assets();
IRequestInteractions(newRequest).asset().checkAssetsMatch(_pmDebt);  // asset-compatible
↪  only
```

Recommendation: Since `setRequest` requires quorum permission, document all the checks guardians should run before approving a new request.

3F: Acknowledged.

Cantina Managed: Acknowledged.

3.3.5 Factory registries are informational, no on-chain check that components were created via Grunt factories

Severity: Low Risk

Context: *(No context files were provided by the reviewer)*

Description: Grunt maintains factory-based deployment registries for most first-class contracts:

- `PositionManagerFactory.isPositionManager(...)`.
- `USCCFundFactory.isFund(...)`.
- `RequestFactory.isRequest(...)`.
- `MorphoBorrowPositionFactory.isBorrowPosition(...)`.
- `TransferGuardFactory.isTransferGuard(...)`.

Nothing downstream actually checks those registries before trusting a configured address. Runtime trust is ”does this address look interface-compatible and asset-compatible?” rather than ”did this address come from the official factory?”.

Concrete gaps:

- **PM provenance exists but Facility ignores it.** `FacilityIntents.createIntent()` accepts any address for `targetAsset` / `guardKey` without checking `PositionManagerFactory.isPositionManager()`.
- **Fund provenance exists but `setFund()` only checks contract-ness and asset/share compatibility.** `FacilityIntents.setFund()` validates `asset()` and `share()` match the PM-reported assets but never consults a fund registry.
- **Request provenance exists but `setRequest()` only checks contract-ness and asset compatibility.** `FacilityIntents.setRequest()` similarly never consults `RequestFactory.isRequest()`.
- **Borrow-position provenance exists but PM module admission only checks owner, assets, and `safeLtv`.** `PositionManagerAdmin` does not verify factory provenance on admitted borrow modules.
- **TransferGuard provenance exists but PM and Facility trust any configured guard that answers the interface.**.
- **PT/YT tokens' provenance does not exist in the RequestFactory.**.

The gap is most directly exploitable at the PM level: a facilitator can call `createIntent()` with a fake PM-shaped contract as `targetAsset`, then call `depositManager()`. Inside `depositManager()`, the Facility approves and transfers collateral to the fake PM via `IPositionManager(pm).deposit()`. A malicious `deposit()` implementation can `transferFrom` the approved collateral to any address, draining the intent's LP funds in one transaction. The same applies to funds, requests, and guards.

This attack becomes realistic if the Facilitator role is compromised, most likely through a backend infrastructure breach. The attacker can route clients to deposit into a malicious intent ID through the compromised backend, with users having no on-chain way to tell a real intent from a fake one.

Recommendation: Enforce the existing `is*()` registry checks at every function that configures a protocol-level address. The registries already exist and are not consulted at bind time:

- `createIntent()` and `updateTarget()` should verify `PositionManagerFactory.isPositionManager(targetAsset)`.
- `setFund()` should verify the fund address against the appropriate fund factory registry.
- `setRequest()` should verify `RequestFactory.isRequest(newRequest)`.
- `addBorrowModule()` should verify `MorphoBorrowPositionFactory.isBorrowPosition(module)`.

- `setTransferGuard()` should verify `TransferGuardFactory.isTransferGuard(guard)`.
- `RequestFactoy` should have mappings that can be used to verify if a PT/YT token is legit and was deployed from that factory contract.

This stops arbitrary contracts from being wired in at the top level, blocking the fake-contract drain attack. It does not provide end-to-end guarantees: Morpho markets, oracles, and tokens are external constructs with no factory registry. Full enforcement at those levels would require additional whitelists for approved markets and tokens, which is a larger design decision. The `is*()` checks are the practical first step.

3F: Acknowledged. We will have potentially multiple factories. It's useful for monitoring for example (using it with Hypernative).

Cantina Managed: Acknowledged.

3.3.6 Exact `preLiquidate()` inputs can be broken by small state changes right before execution

Severity: Low Risk

Context: *(No context files were provided by the reviewer)*

Description: `preLiquidate()` trusts exact caller-provided inputs against a live Morpho position that other users can change before the transaction lands.

There are two branches:

Exact-share branch: If the liquidator asks to repay an exact number of borrow shares, a third party can front-run with a small direct Morpho repay and reduce the live share balance first. The old exact input then causes `_computeSeizedAssets()` to compute against a smaller share count, which can revert or produce an unexpected result.

Exact-collateral branch: If the liquidator asks to seize an exact amount of collateral, a third party can front-run with a small native Morpho liquidation and reduce the live collateral first. The old exact collateral input then causes `_computeRepaidShares()` to compute against a smaller collateral count, reverting the transaction.

```
// src/borrow/MorphoBorrowPosition.sol:275-308
function preLiquidate(address borrower, uint256 seizedAssets, uint256 repaidShares,
↪  bytes calldata data)
    external
    returns (uint256, uint256)
{
//...
    if (seizedAssets > 0) {
        repaidShares = _computeRepaidShares(position, market, _storage.marketParams,
        ↪  seizedAssets);
    } else {
        seizedAssets = _computeSeizedAssets(position, market, _storage.marketParams,
        ↪  repaidShares);
    }
//...
}
```

`_computeRepaidShares()` computes proportional shares from `seizedAssets / collateral`. If a front-runner has reduced the live collateral below `seizedAssets`, the computation reverts:

```
// src/borrow/MorphoBorrowPosition.sol:314-340
function _computeRepaidShares(..., uint256 seizedAssets) internal view returns (uint256
↪  repaidShares) {
    uint256 borrowShares = uint256(position.borrowShares);
    uint256 collateral = uint256(position.collateral);
    uint256 proportional = seizedAssets.mulDivUp(borrowShares, collateral);
//...
}
```

`_computeSeizedAssets()` similarly computes from `repaidShares / borrowShares`. If a front-runner has reduced the live borrow shares below `repaidShares`, the computation reverts:

```
// src/borrow/MorphoBorrowPosition.sol:345-375
function _computeSeizedAssets(..., uint256 repaidShares) internal view returns (uint256
↪  seizedAssets) {
    uint256 borrowShares = uint256(position.borrowShares);
    uint256 collateral = uint256(position.collateral);
    uint256 proportional = repaidShares.mulDiv(collateral, borrowShares);
    uint256 remainingBorrowShares = borrowShares - repaidShares;
//...
}
```

The result is a liquidation griefing bug: the liquidation must be re-quoted and retried. A third party can repeat this cheaply with dust operations, keeping the unhealthy position alive while debt keeps growing.

Recommendation: There are two ways to handle this:

- **Option 1 - Document and accept:** Document that exact-input liquidation values can be front-run on public mempools, and recommend callers use private mempools or re-quote on revert.
- **Option 2 - Code fix:** Clamp the caller's exact inputs to the live position state so a small front-run reduction does not revert the transaction. For the exact-share branch, cap `repaidShares` to `min(repaidShares, position.borrowShares)`. For the exact-collateral branch, cap `seizedAssets` to `min(seizedAssets, position.collateral)`. This way the liquidation still goes through at the reduced amount rather than reverting entirely.

3F: Acknowledged.

Cantina Managed: Acknowledged.

3.3.7 Deposit collateral routes to supplyQueue[0], assuming uncapped supply and requiring manual rebalance

Severity: Low Risk

Context: *(No context files were provided by the reviewer)*

Description: All three collateral deposit paths unconditionally route to `supplyQueue[0]` with no cap guard:

Pure-collateral deposits (`debt == 0`) in `PositionManagerLP.sol:63-67` :

```
// PositionManagerLP.sol:63-67
if (debt == 0) {
    if (collateral > 0) {
        if (_storage.supplyQueue.length == 0) revert LibManagerErrors.EmptySupplyQueue();
        _storage.supplyQueue[0].position.supply(_storage.metadata.collateralAsset,
        ↪  collateral);
    }
}
```

Leftover collateral after the borrow loop in `LibOperations.sol:88-90` :

```
// LibOperations.sol:88-90
if (remainingCollateral > 0) {
    _storage.supplyQueue[0].position.supply(_storage.metadata.collateralAsset,
    ↪  remainingCollateral);
}
```

Per-borrow collateral inside the loop: each `position.supply(collateralNeeded)` call (`LibExecutor.sol:57`) forwards exactly what the LTV formula computes with no cap guard.

1. No cap check on `supplyCollateral`.

All three paths call `supplyCollateral` without checking whether the target position can accept the full amount. This works now because Morpho Blue has no collateral cap. But `IBorrowPosition` is generic, and other integrations might have per-market or per-position caps. If a future integration adds a cap and a deposit hits it, `supplyCollateral` reverts and the entire `deposit()` or `depositManager()` call fails. Deposits would be stuck on that position until the curator reconfigures the supply queue to route around it.

2. Collateral always goes to position 0, rebalance required to spread it.

The PM never spreads collateral across positions on its own. All of it ends up in `supplyQueue[0]`: pure-collateral deposits go there directly, and the borrow loop sends whatever is left over there too. If the curator wants collateral spread across multiple positions, they need to call `rebalance()` after every deposit. If a rebalance cooldown is configured (`PositionManagerRebalancing.sol:54-63`), position 0 stays over-collateralized for the full cooldown window after each deposit.

Recommendation:

- Document that `processDeposit` and the pure-collateral path assume `supplyCollateral` never reverts due to a cap. Any future `IBorrowPosition` implementation must also be uncapped, or the routing logic needs to handle partial fills.
- Document that depositing does not spread collateral across positions. Curators should call `rebalance()` after deposits if even collateral distribution is needed.
- If even distribution is a first-class goal, consider routing leftover collateral proportionally across all supply queue entries instead of always sending it to index 0.

3F: Acknowledged.

Cantina Managed: Acknowledged.

3.3.8 ERC-6909 per-intent approve is unusable: withdraw and claim only accept global operators

Severity: Low Risk

Context: *(No context files were provided by the reviewer)*

Description: `FacilityLP` inherits Solady's `ERC6909`, which exposes two delegation mechanisms: `approve(spender, id, amount)` for per-token-ID allowances and `setOperator(operator, approved)` for a global all-or-nothing flag. The Facility's LP shares are indexed by intent ID, so `approve` looks like the natural way for an LP to delegate access to a specific intent without giving a third party control over all of their positions.

However, `withdraw()` and `claim()` both route through `_withdrawalLpChecks()`, which only checks the global operator flag:

```
// FacilityLP.sol:149-152
if (from != msg.sender && !isOperator(from, msg.sender)) {
    revert InsufficientPermission();
}
```

The per-ID allowance set by `approve()` is never read or used in this path. A user who calls `approve(bob, intentId, amount)` expects bob to be able to act on that specific intent. Bob cannot: the call reverts with `InsufficientPermission()` regardless of the allowance amount.

`setOperator(bob, true)` is the only thing that works, but it gives bob access to all your intents, present and future.

Recommendation: There are two ways to handle this:

- **Option 1 - Document and accept:** Document that ERC-6909 `approve()` is a no-op for withdrawal purposes and direct users to `setOperator()` instead. Note that `setOperator()` gives the delegate unrestricted access to withdraw and claim from every intent the user holds, including intents created in the future. A user who wants to allow a third party (such as a relayer or portfolio manager) to act

on one specific intent has no safe way to do so today: they must either act themselves or hand over global control.

- **Option 2 - Code fix:** Use the per-ID allowance in `_withdrawalLpChecks()` alongside the operator check. Note that Solady's `ERC6909` does not expose an internal `_decreaseAllowance` helper, so this requires a custom override to read and write the allowance slot directly:

```
if (from != msg.sender && !isOperator(from, msg.sender)) {
    uint256 allowed = allowance(from, msg.sender, id);
    if (allowed < amount) revert InsufficientPermission();
    if (allowed != type(uint256).max) {
        _decreaseAllowance(from, msg.sender, id, amount);
    }
}
```

3F: Acknowledged. Unless at some point it starts to be a problem for integrations, we will think about doing it.

Cantina Managed: Acknowledged.

3.3.9 Guardian signatures have no nonce: old approvals stay executable after guardians move on

Severity: Low Risk

Context: *(No context files were provided by the reviewer)*

Description: The three guardian-quorum operations `setFund()`, `setRequest()`, and `swap()` each produce an EIP-712 digest that binds only the operational parameters and a deadline. None include a nonce or any reference to the intent's current configuration:

```
// setFund - FacilityIntents.sol:168
_hashTypedData(keccak256(abi.encode(SET_FUND_PARAMS_TYPEHASH, id, newFund, deadline)))

// setRequest - FacilityIntents.sol:220
_hashTypedData(keccak256(abi.encode(SET_REQUEST_PARAMS_TYPEHASH, id, newRequest,
↪  deadline)))

// swap - FacilitySwap.sol:88
_hashTypedData(keccak256(abi.encode(SWAP_PARAMS_TYPEHASH, params)))
// SwapParams: (id1, token1, id2, token2, amount1, amount2, deadline)
```

`checkDigest()` burns a digest on first submission, so the same digest cannot be submitted twice. But each set of parameters produces a distinct digest, and all unused digests remain independently executable until their individual deadlines.

setFund / setRequest: stale approval survives intent re-configuration.

Guardians approve `setFund(id, F1, deadline)` for a specific fund. Something changes, and guardians sign a fresh `setFund(id, F2, deadline2)` to replace it. Both digests are now live. The facilitator can execute F2 first, then submit the old F1 approval later, binding a fund that the guardians explicitly moved away from. The intent's `guardKey`, old fund/request bindings, and any intermediate changes are invisible to the signed message.

swap: outdated exchange rate can be executed after guardians corrected it.

Guardians approve a swap between intent 1 and intent 2 at a rate of 100 USDC for 1 WETH. Market conditions change. Guardians sign a corrected swap at 100 USDC for 2 WETH to reflect fair value. Both digests remain valid. The facilitator executes the corrected swap first, then submits the original approval. Intent 1 gives away another 100 USDC at the stale rate, and intent 2 receives 1 WETH it was never supposed to get under the updated terms.

Recommendation:

- Add a per-intent nonce to the Facility and include it in the signed payload for `setFund()`, `setRequest()`, and `swap()`. Increment the nonce on every change that affects guardian-relevant

context, at minimum on each successful `setFund()`, `setRequest()`, and `swap()` execution. This ties every approval to one specific state transition rather than being valid across any future state within the deadline window.

- Alternatively, document clearly that any signed digest stays executable by the facilitator until its deadline, even after guardians have agreed on new terms. Guardians should use short deadlines and treat each signing as a live authorization that cannot be revoked once issued.

3F: Acknowledged. We will avoid long running signatures. We must be careful.

Cantina Managed: Acknowledged.

3.3.10 No slippage check at Fund commit: funds can leave the Facility at a rate worse than what was agreed at create

Severity: Low Risk

Context: *(No context files were provided by the reviewer)*

Description: `FacilityFunds.create()` accepts a `minAmountOut` parameter and stores it as `order.output`:

```
// FacilityFunds.sol:67
order = Order({ ..., output: minAmountOut, ... });
```

The slippage bound is only enforced here. By the time the facilitator calls `FacilityFunds.commit()`, which is when funds actually leave the Facility to the third-party fund, the exchange rate may have moved. `commit()` passes the order to the fund but does not re-check the current quoted output against `order.output`:

```
// FacilityFunds.sol:113
(, uint256 _committedAmount) = IFund(_fund).commit(_order);
if (_committedAmount != _order.input) revert ...; // input integrity only, no output
↪  check
```

Neither `USCCFund.commit()` nor `ParetoFund.commit()` reads `order.output`. Funds are transferred to the third party with no on-chain guarantee the output meets the floor set at order creation.

`commit()` is the last point where `cancel()` is still straightforward, since the funds have not yet left. After commit, most funds enter an async processing state and cancellation requires a separate recovery path. Nothing forces the facilitator to cancel when the rate moves against the intent, so LP funds can get committed at a worse rate than what was quoted at `create()`.

Recommendation: There are two ways to handle this:

- **Option 1 - Document and accept:** Document that `order.output` is only enforced at `create()` time and that the facilitator bears responsibility for rate drift between create and commit.
- **Option 2 - Code fix:** Before calling `IFund(_fund).commit(_order)` in `FacilityFunds.commit()`, query the current exchange rate from the fund and verify the expected output still meets `order.output`. If the current quoted output is below the floor, revert so the facilitator is forced to either cancel the order or update the intent with a new `create()` at the current rate.

3F: Acknowledged. It depends on the fund we are interacting with. At the moment, with the current implementations, we are executing create and commit almost at the same time. Also, checking the fund rate (using totalAssets?) is not representative of the exact output you can get (a lot of things happen offchain with rwas). If there is an async process between the two actions, we can expect the facilitator to check the price.

Cantina Managed: Acknowledged.

3.3.11 `PositionManager` share price is temporarily wrong between token transfer and `_settleShares`, enabling read-only reentrancy

Severity: Low Risk

Context: *(No context files were provided by the reviewer)*

Description: Both `deposit()` and `withdraw()` in `PositionManagerLP` do an external token transfer before calling `_settleShares()`. This creates a window where `totalAssets()` and `totalSupply()` are out of sync, and any external call made during that window reads a share price that doesn't reflect the final state.

deposit() ordering (`PositionManagerLP.sol:54-82`):

```
1. _accrueFees() / totalSupply()        // snapshot before
2. pull collateral from caller
3. processDeposit(collateral, debt)     // NAV increases: collateral supplied, debt
↪  borrowed
4. safeTransfer(debt, msg.sender)       // ← external call: totalAssets() is higher,
↪  totalSupply() is still old
5. _settleShares(...)                   // shares minted here, closing the window
```

During step 4, if the debt token has a transfer hook, the hook fires while `totalAssets() / totalSupply()` is inflated. Any external contract reading this ratio prices PM shares too high.

withdraw() ordering (`PositionManagerLP.sol:101-119`):

```
1. _accrueFees() / totalSupply()        // snapshot before
2. pull debt from caller
3. processWithdrawal(collateral, debt)  // NAV decreases: collateral withdrawn, debt
↪  repaid
4. safeTransfer(collateral, msg.sender) // ← external call: totalAssets() is lower,
↪  totalSupply() is still old
5. _settleShares(...)                   // shares burned here, closing the window
```

During step 4, if the collateral token has a transfer hook, the hook fires while `totalAssets() / totalSupply()` is deflated.

The exposed view functions are `PositionManager.totalAssets()` and `PositionManager.totalSupply()`. If a 3rd party integrator prices PM shares from this ratio, an attacker can use the temporary misprice:

- **On deposit**: the inflated price lets an attacker overborrow against PM shares used as collateral in the external contract.
- **On withdraw**: the deflated price can trigger liquidations of PM share positions that should not have happened, or let an attacker buy PM shares at a price that is too low.

The `nonReentrant` guard on `deposit()` and `withdraw()` blocks reentry into those functions but does not stop the view functions from being read during the external call window.

Recommendation:

- Move `_settleShares()` before the external token transfer so supply and assets are always in sync when control leaves the contract. This closes the window without breaking any integration or callback that reads `totalAssets()` / `totalSupply()`.
- Optionally, as additional defense-in-depth, add a reentrancy lock to `totalAssets()` and `totalSupply()` so they revert if called while `deposit()` or `withdraw()` is active (same pattern Curve pools use). Note that this breaks any callback or integration that reads those views mid-operation, so only do this if you are willing to treat those views as intentionally unavailable during transfers.

3F: Fixed in PR 183.

Cantina Managed: Fix verified.

3.3.12 Blocklisted approved spenders and operators can still move guarded `Facility` and `PositionManager` shares

Severity: Low Risk

Context: *(No context files were provided by the reviewer)*

Description: `TransferGuard.canTransfer()` only checks the token, the sender, and the receiver. It never sees the actual delegated caller:

```
function canTransfer(address token, address from, address to, uint256 amount) external
↪   view returns (bool) {
    // ...
    if (from != address(0) && !_isAllowed($, from, config.whitelist)) return false;
    if (to != address(0) && !_isAllowed($, to, config.whitelist)) return false;
    return true;
}
```

Both `Facility._beforeTokenTransfer()` and `PositionManager._beforeTokenTransfer()` forward only `from` and `to` into the guard, so the actual `msg.sender` of a delegated call is never seen by the guard.

Once a user has approved a spender or granted global operator rights, blocking that caller afterward has no effect. The blocked caller can still transfer on behalf of others as long as `from` and `to` are not themselves blocked.

In a router scenario, for example, if a router that handles shares becomes malicious, it cannot be blocked from operating on behalf of users who approved it. Users have to revoke their own approvals.

Recommendation: There are two ways to handle this:

- **Option 1 - Document and accept:** Document that the guard only checks from/to addresses and does not see the delegated caller (`msg.sender`). Users must revoke approvals to untrusted spenders.
- **Option 2 - Code fix:** Pass the delegated caller into the guard and enforce policy on that address as well.

3F: Acknowledged.

Cantina Managed: Acknowledged.

3.3.13 Morpho borrow module oracle revert causes permanent PM DoS through `_accrueFees()` and bricks `removeBorrowModule()`

Severity: Low Risk

Context: *(No context files were provided by the reviewer)*

Description: `LibView.totalAssets()` loops over every live borrow module and calls `totalCollateralQuoted()` with no fault isolation:

```
// LibView.sol:56-64
function totalAssets(PositionManagerStorageData storage ps) internal view returns
↪   (uint256 amount) {
    address[] memory modules = ps.borrowModules.values();
    uint256 modulesLength = modules.length;
    for (uint256 i = 0; i < modulesLength; ++i) {
        uint256 collateral = IBorrowPosition(modules[i]).totalCollateralQuoted();
        uint256 debt = IBorrowPosition(modules[i]).totalBorrowed();
        amount += collateral.zeroFloorSub(debt);
    }
}
```

`IBorrowPosition` does not require an oracle. The oracle dependency is specific to the current Morpho implementation. `MorphoBorrowPosition.totalCollateralQuoted()` makes a bare external oracle call:

```
// MorphoBorrowPosition.sol:458-462
function totalCollateralQuoted() external view override returns (uint256) {
    BorrowPositionStorage storage _storage = _borrowPositionStorage();
    return uint256(MORPHO.position(_storage.marketId, address(this)).collateral)
        .mulDiv(IOracle(_storage.marketParams.oracle).price(), ORACLE_PRICE_SCALE);
}
```

If any live borrow module's oracle reverts, `totalAssets()` reverts. Every state-mutating PM path calls `_accrueFees()` first, and `_accrueFees()` always recomputes `totalAssets()`. The result is a full PM DoS while the oracle is down: deposits, withdrawals, burns, rebalances, and fee updates all revert.

The recovery path is also blocked: `removeBorrowModule()` calls `_accrueFees()` before removing the broken module from `borrowModules`:

```
// PositionManagerAdmin.sol:60-79
function removeBorrowModule(address module) external override onlyOwner {
    _accrueFees();
//...
    if (!_storage.borrowModules.remove(module)) { ... }
    _storage.updateSnapshot();
}
```

The owner cannot use the normal on-chain module-removal path to detach the broken sleeve while its oracle is reverting. `LibStorage.updateSnapshot()` also calls `LibView.totalAssets()`, so even post-removal accounting helpers share the same bug.

Recommendation: Add an owner-only emergency module-removal path that does not require `_accrueFees()` to succeed first.

3F: Acknowledged.

It's not only the problem, the morpho market is also bricked.

If it happens we can upgrade the PositionManager contract, since we can expect this scenario to be very unusual. In the history of DeFi, major oracle networks like Chainlink and Pyth have never experienced a prolonged outage lasting days or weeks that left protocols without any usable price data for an extended period. The longest documented disruptions were relatively short: Chainlink's ETH/USD feed stalled for approximately six hours in March 2020 due to extreme Ethereum gas congestion, while MakerDAO's oracles suffered similar temporary update delays during the same "Black Thursday" event (on the order of hours rather than days). More recent incidents, such as Pyth's brief ~40-minute downtime in 2025, were even shorter and quickly resolved. Overall, these events remained limited in duration.

Cantina Managed: Acknowledged.

3.3.14 USCCFund only validates the fund, not the eventual restricted-wrapper receiver

Severity: Low Risk

Context: *(No context files were provided by the reviewer)*

Description: `USCCFund.create(...)` and `USCCFund.commit(...)` only check whether the fund itself is currently Superstate-allowlisted:

```
// USCCFund.sol:176-178 / 232-234
if (!ISuperstateToken(USCC).isAllowed(address(this))) {
  revert LibFundsErrors.NotAllowedSuperstate();
}
```

After the transfer-guard readaptation, completing an order no longer depends only on the fund being allowed. Both `recover(...)` and `unlock(...)` mint `wUSCC` to `order.receiver`, which is forced to `msg.sender` during `create(...)`:

```
// USCCFund.sol:268-270 / 293-295
USCC.safeApproveWithRetry(WUSCC, _amount);
IWrappedAsset(WUSCC).mint(order.receiver, _amount);
```

That mint now routes through the restricted wrapper hook in `WrappedAsset._beforeTokenTransfer(...)`, and the Superstate-specific override in `SuperstateRestrictedWrappedAsset.isAllowed(...)` makes that hook depend on `USCC.isAllowed(account)`.

An order can pass `create()` and `commit()`, send USDC into the offchain Superstate leg, receive USCC back to the fund, and then get permanently stuck when `unlock()` or `recover()` tries to mint restricted `wUSCC` to a receiver that is no longer allowed. This is reachable in two ways:

- The depositor/receiver was never eligible to receive restricted `wUSCC`, but the fund accepted the order because it only checked `address(this)`.
- The receiver was eligible when the async order was committed but loses Superstate eligibility before `unlock()` or `recover()`.

Note: currently the receiver is always hardcoded to the Facility address which should be whitelisted by Superstate, but if that changes in the future this issue becomes more severe.

Recommendation: There are two ways to handle this:

- **Option 1 - Document and accept:** Document the Superstate allowlist dependency and the assumption that `order.receiver` must stay allowlisted through the full order flow. Currently the receiver is always the Facility address which is whitelisted, so this is not an active risk.
- **Option 2 - Code fix:** Reject `create()` and `commit()` unless the eventual `order.receiver` is currently `ISuperstateToken(USCC).isAllowed(...)`. Add a rescue path for returned USCC that does not depend on minting restricted wrapper shares to a now-blocked receiver.

3F: Fixed in PR 182.

Cantina Managed: PR 182 is a partial fix: it adds the receiver allowlist check at `create()` and `commit()` time but does not add a rescue path for returned USCC to a now-unallowed receiver. Since losing Superstate eligibility is not equivalent to being blocklisted, the intended recovery is to re-allow the receiver rather than implement a special rescue path.

3.3.15 PM fee shares are minted against the pre-fee base and underpay configured fees

Severity: Low Risk

Context: *(No context files were provided by the reviewer)*

Description: `PositionManagerBase._pendingFees()` computes management and performance fees in asset terms, then converts those fee assets to shares with `convertToShares(totalSupply_, totalAssets_, virtualShareOffset_, false)` against the untouched pre-fee base:

```
// PositionManagerBase.sol:71-84
managementFeeAssets = totalAssets_.mulDiv(fd.managementFee * elapsed, BPS *
↪  SECONDS_PER_YEAR);
if (managementFeeAssets > 0) {
    managementFeeShares +=
        managementFeeAssets.convertToShares(totalSupply_, totalAssets_,
        ↪  virtualShareOffset_, false);
}
// ...
uint256 performanceFeeAssets = gains.mulDiv(fd.performanceFee, BPS);
if (performanceFeeAssets > 0) {
    performanceFeeShares +=
```

```
        performanceFeeAssets.convertToShares(totalSupply_, totalAssets_,
        ↪  virtualShareOffset_, false);
}
```

`_pendingFees()` first carves `managementFeeAssets` and `performanceFeeAssets` out of `totalAssets_`, then calls `convertToShares(..., totalSupply_, totalAssets_, ...)` as if LPs still owned the full `totalAssets_` base. That's the wrong number. If `feeAssets` are meant to become a claim on the existing asset pool, the minted fee shares must be priced against the LP-owned remainder, not against the unadjusted pool that still includes the fee slice itself. The conversion should solve for shares against `totalAssets_ - feeAssets` (or the equivalent post-mint equation), not against the untouched `totalAssets_`.

Because `_accrueFees()` then mints those underpriced shares, the fee recipient receives fewer shares than needed for those shares to redeem back to the originally computed fee assets. The configured management/performance fee percentages end up understated.

This is LP-favorable rather than a user loss, but the PM advertises management and performance fees in asset-space terms and the minted share payout is smaller than those computed fee assets.

Concrete example: deposit `10_000e18`, accrue a 1-year 2% management fee. `_pendingFees()` reports `200e18` fee assets. After accrual the fee recipient burns those shares and receives only `~196.08e18` collateral, about 2% short of the advertised fee.

Recommendation: Do not convert fee assets with `convertToShares(totalSupply_, totalAssets_, ...)` directly after deriving those fee assets from the same `totalAssets_`. Instead, compute the combined fee assets first and solve the minted share amount against the fee-adjusted base, so the newly minted shares represent exactly that fee slice of the pool.

3F: Fixed in PR 175.

Cantina Managed: Fix verified.

3.3.16 `checkCollateralAllowed` can brick PM fee accrual and fee rotation

Severity: Low Risk

Context: *(No context files were provided by the reviewer)*

Description: `PositionManagerBase._accrueFees()` mints newly accrued fee shares directly to the current `feeRecipient`:

```
// PositionManagerBase.sol:98-104
uint256 feeShares = managementFeeShares + performanceFeeShares;
if (feeShares > 0) {
    address feeRecipient = LibStorage.positionManagerStorage().feeData.feeRecipient;
    _mint(feeRecipient, feeShares);
    emit IPositionManagerLP.FeesAccrued(feeRecipient, feeShares);
}
```

That mint routes through `PositionManager._beforeTokenTransfer(...)`, which checks the live `TransferGuard` for every mint, burn, and transfer. After the transfer-guard readaptation, the owner can also enable `checkCollateralAllowed`, which makes the guard additionally check the collateral wrapper's `isAllowed(account, amount)` hook for each non-null party:

```
// TransferGuard.sol:165-170
if (config.checkCollateralAllowed) {
    (address collateral,) = IPositionManager(token).assets();
    if (from != address(0) && !IWrappedAsset(collateral).isAllowed(from, amount)) return
    ↪  false;
    if (to != address(0) && !IWrappedAsset(collateral).isAllowed(to, amount)) return
    ↪  false;
}
```

Once that flag is on, fee accrual only works if the collateral wrapper keeps accepting the current `feeRecipient`. If the collateral wrapper returns `false` for the fee recipient, `_mint(feeRecipient, feeShares)` reverts with `TransferBlocked` and the PM deadlocks at the accrual step.

That deadlock affects every PM entrypoint because `_accrueFees()` runs first in all of them:

- `PositionManagerLP.deposit(...)`.
- `PositionManagerLP.withdraw(...)`.
- `PositionManagerLP.burn(...)`.

The normal recovery path is also blocked: `PositionManagerAdmin.setFeeData(...)` calls `_accrueFees()` first, before rotating the recipient, so the owner cannot switch away from the now-disallowed fee recipient once fees are pending.

This is a real problem on `wUSCC`-backed PMs. `SuperstateRestrictedWrappedAsset.isAllowed(...)` ignores `amount` and delegates straight to `USCC.isAllowed(account)`, so enabling `checkCollateralAllowed` means Superstate must keep the PM fee recipient allowlisted as long as fee minting can occur. No `TransferGuard` blocklist or whitelist change is needed to trigger the freeze; collateral-layer allowlist drift is enough.

Recommendation: In addition to issue `addBorrowModule` and `removeBorrowModule` cause PM NAV jumps that can be sandwiched's "rotate before blocking" recommendation, note that when `checkCollateralAllowed` is enabled the rotation must also be coordinated with the underlying collateral authority. For a `wUSCC`-backed PM, the new fee recipient must be whitelisted on Superstate **before** `setFeeData()` is called, and the current fee recipient must remain on Superstate's allowlist until pending fees are cleared, otherwise `_accrueFees()` inside `setFeeData()` reverts and the deadlock continues.

3F: Acknowledged.

Cantina Managed: Acknowledged.

3.3.17 `addBorrowModule` and `removeBorrowModule` cause PM NAV jumps that can be sandwiched

Severity: Low Risk

Context: *(No context files were provided by the reviewer)*

Description: `addBorrowModule()` and `removeBorrowModule()` in `PositionManagerAdmin.sol` change the `borrowModules` set, and `LibView.totalAssets()` loops over that set to compute PM value:

```
function totalAssets(PositionManagerStorageData storage ps) internal view returns
↪   (uint256 amount) {
    address[] memory modules = ps.borrowModules.values();
    uint256 modulesLength = modules.length;
    for (uint256 i = 0; i < modulesLength; ++i) {
        uint256 collateral = IBorrowPosition(modules[i]).totalCollateralQuoted();
        uint256 debt = IBorrowPosition(modules[i]).totalBorrowed();
        amount += collateral.zeroFloorSub(debt);
    }
}
```

Both admin functions follow this pattern:

```
function addBorrowModule(address module) external override onlyOwner {
    _accrueFees();
//...
    if (!_storage.borrowModules.add(module)) { revert ... }
    _storage.updateSnapshot();
//...
}
```

```
function removeBorrowModule(address module) external override onlyOwner {
    _accrueFees();
//...
    if (!_storage.borrowModules.remove(module)) { revert ... }
    _storage.updateSnapshot();
//...
}
```

Neither function requires the module to be economically flat (zero collateral, zero debt) before adding or removing it. The moment the set changes, `totalAssets()` jumps by the module's net equity in one transaction, and anything that prices off it (`_settleShares()` , fee accrual, rebalance checks) sees the jump immediately.

Example: PM has sleeve A (net equity 40) and sleeve B (net equity 50), `totalAssets = 90` , `totalSupply = 100` , share price = 0.9. Owner calls `removeBorrowModule(A)` . After removal, `totalAssets = 50` , share price = 0.5. That is a 44% share price drop in one transaction. `addBorrowModule(M)` on a pre-loaded module produces the same thing in reverse.

Both functions are `onlyOwner` , which controls who submits the transaction but does nothing about who sees it. The admin transaction sits in the public mempool and the jump size is deterministic (it is the net equity of the module being added or removed, readable from public state). Anyone watching can line their own transactions up around it:

- **Around** `removeBorrowModule(M)` : frontrun with `burn()` / `withdraw()` at the pre-drop share price to exit before the pool shrinks. Optionally backrun with `deposit()` at the new lower price.
- **Around** `addBorrowModule(M)` : frontrun with `deposit()` at the pre-jump share price, then backrun with `burn()` / `withdraw()` at the new higher price.

Recommendation: No clean fix is available. One option is to require the modules to be flat (i.e., `totalCollateral == 0 && totalBorrowed == 0`) before adding or removing, but that may not be acceptable in all cases.

3F: Acknowledged.

Cantina Managed: Acknowledged.

3.3.18 Some fund adapters accept nonzero dust orders whose computed output is zero

Severity: Low Risk

Context: *(No context files were provided by the reviewer)*

Description: The fund adapters check slippage in `create()` by computing an expected output at the current oracle or vault rate and reverting only when `order.output` is way below that value. When the expected output itself rounds to `0` , the slippage check is a no-op and `order.output = 0` passes even though the order has nonzero input. This is a dust-order bug, not a drain primitive: it lets value get burned or stranded on dust-sized orders.

USCCFund deposit path

`USCCFund._validateOutput()` computes `expectedOutput = order.input * 1e6 / price` . If that floors to `0` , `create()` accepts `order.output = 0` . `commit()` still sends real USDC to the Superstate recipient. `_state()` then checks `balance >= order.output` , which with `order.output == 0` is always true, so the order moves straight to `UNLOCKING` . `unlock()` finalizes with `_amount = 0` , and `WrappedAsset.mint()` accepts zero. The input for that order is gone.

CentrifugeFund zero-output requests

`CentrifugeFund.create()` has the same pattern via `convertToShares()` / `convertToAssets()` . If those round to `0` , `order.output = 0` passes and `commit()` still fires off the real async request upstream. `_state()` only moves to `UNLOCKING` when `maxMint > 0` (deposits) or `maxWithdraw > 0`

(redeems). If the upstream dust request settles with nothing claimable, the local order never settles and stays stuck until an operator calls `forceEnd()`.

Recommendation:

- Reject any nonzero order in `create()` when the computed expected output is `0`.
- As a second line of defense, require a strictly positive claimable amount before reporting `UNLOCKING` on the async adapters.
- On `USCCFund`, make `unlock()` revert on zero-amount finalization so a zero-return path cannot silently close.

3F: Acknowledged. Even if the output is 0, the commitment is close to 0. If the output at 0 creates errors on the Facility side, maybe we can think about it.

Cantina Managed: Acknowledged.

3.3.19 `Facility.pauseFor()` does not stop Request, WrappedAsset, or fund adapter state changes

Severity: Low Risk

Context: *(No context files were provided by the reviewer)*

Description: The only pause mechanism in the codebase is `Facility.pauseFor()` plus per-token `TransferGuard` pauses on PM / Facility share tokens. None of the standalone value-holding contracts under `src/request` or `src/funds` check any pause flag.

Pausing the Facility does not freeze direct state changes on bound `Request` contracts, the shared `WrappedAsset`, or the fund adapters (`USCCFund`, `CentrifugeFund`, `ParetoFund`).

On `Request`, the owner and consumer can still call `authorizeMinting()`, `consume()`, and `setRepaid()`. Authorized minters can still call `mint()`. Anyone can still `repay()`. PT/YT holders can still redeem through `burnAll()` once withdrawals are open.

On `WrappedAsset`, users can still `mint()` and `burn()` with no pause guard.

On fund adapters, privileged actors can still progress or finalize orders directly through `create()`, `commit()`, `unlock()`, `recover()`, and `resolve()`:

- `USCCFund` has no pause guard on any order method.
- `CentrifugeFund` has no pause guard on any order method.
- `ParetoFund` has no pause guard on any order method.

During an incident, responders might think everything is frozen while Request, WrappedAsset, and fund state keeps changing under them.

Recommendation: There are two ways to handle this:

Option 1 - Document and accept: Document clearly that `Facility.pauseFor()` is not a system-wide stop and should not be treated as one in incident-response runbooks. Make sure operators know that Request, WrappedAsset, and fund adapters remain fully active when the Facility is paused.

Option 2 - Code fix: Add a shared pause source that `Request`, `WrappedAsset`, and fund adapters all check before state-changing operations. If some flows must stay available during an incident (e.g. PT/YT redemptions), split pause modes explicitly instead of relying on the Facility pause as the only emergency control.

3F: Acknowledged. Indeed, it's not stopping everything. But the facility is the central piece of the protocol, stopping most of the flows.

Cantina Managed: Acknowledged.

3.3.20 PM burn shows zero debt but final exit still reverts because of dust borrow shares

Severity: Low Risk

Context: *(No context files were provided by the reviewer)*

Description: `MorphoBorrowPosition.totalBorrowed()` converts raw Morpho borrow shares to asset units using `toAssetsDown` (rounds down):

```
// src/borrow/MorphoBorrowPosition.sol:435-446
function totalBorrowed() external view override returns (uint256) {
//...
    return uint256(_pos.borrowShares).toAssetsDown(_totalBorrowAssets,
    ↪  _totalBorrowShares);
}
```

After interest kicks in and the borrow-share exchange rate is no longer a clean integer, repaying through the normal PM path using `totalBorrowed()` can leave a tiny leftover `borrowShares`. That leftover rounds down to 0 in `totalBorrowed()`, so `LibView.debtAmount()` also returns 0, and `PositionManagerLP.burn()` quotes zero debt for a full exit.

But `availableCollateral()` uses `toAssetsUp` (rounds up) when converting the same leftover:

```
// src/borrow/MorphoBorrowPosition.sol:512-524
if (_pos.borrowShares == 0) return uint256(_pos.collateral);
//...
uint256(_pos.borrowShares).toAssetsUp(_totalBorrowAssets, _totalBorrowShares),
```

So the leftover still matters when the code actually runs. The PM tells the user they can exit with zero debt, but `availableCollateral()` still sees the nonzero borrow shares and blocks full collateral release.

When multiple users burn one by one, the earlier ones get quoted `debtIn == 0` and exit fine. The last user trying to close out the PM hits the revert because `availableCollateral()` still sees the leftover borrow shares, so the last user cannot exit cleanly.

Recommendation: Document that after interest kicks in, a tiny borrow-share leftover can remain after repaying `totalBorrowed()`. The last PM holder may need to repay 1 extra wei of debt to clear it before burning.

3F: Acknowledged.

Cantina Managed: Acknowledged.

3.3.21 Anyone can repay Morpho debt on behalf of a PM sleeve, and `_accrueFees()` counts it as PM performance

Severity: Low Risk

Context: *(No context files were provided by the reviewer)*

Description: Morpho Blue lets anyone repay debt on behalf of any borrower. `Morpho.repay()` does not check who the caller is:

```
Morpho.repay(marketParams, assets, 0, address(borrowPosition), "")
```

The PM immediately sees the lower debt in `LibView.totalAssets()` at `LibView.sol:56-64`, but `lastTotalAssets` is still the old value until the next `_accrueFees()` call. When that call runs, the NAV jump includes the externally donated debt reduction, and performance fee shares get minted on value the PM never earned.

The fee math at `PositionManagerBase.sol:51-87`:

```
if (fd.performanceFee > 0 && totalAssets_ > _lastTotalAssets + managementFeeAssets) {
    uint256 gains = totalAssets_ - _lastTotalAssets - managementFeeAssets;
    // performance fee minted on `gains`, regardless of source
}
```

`_pendingFees()` charges performance fees on any positive `totalAssets - lastTotalAssets` delta, no matter where it came from. An outsider repaying sleeve debt looks exactly the same as a real PM strategy gain at this layer.

So a third-party debt donation meant to help the PM gets partially skimmed as performance fees by the fee recipient. If the fee recipient or a large PM shareholder coordinates the donation, they can turn part of the donated debt relief into fee shares for themselves.

Recommendation: Document that direct `Morpho.repay(onBehalf)` calls to a PM sleeve cause performance fees to be minted on value the PM did not earn. Anyone donating debt relief should be aware the fee recipient takes a cut.

3F: Acknowledged.

Cantina Managed: Acknowledged.

3.3.22 `ParetoFund` blocks redeems that the upstream Pareto vault would allow when `keyringAllowWithdraw` is on

Severity: Low Risk

Context: *(No context files were provided by the reviewer)*

Description: `ParetoFund.create()` always checks `isWalletAllowed(address(this))` for every order mode, including redeems. `ParetoFund.commit()` does the same check again:

```
// ParetoFund.sol:151-152
if (!IIdleCDOEpochVariant($.vault).isWalletAllowed(address(this))) {
    revert LibFundsErrors.NotAllowedByFund();
}
```

This is stricter than what the upstream Pareto vault actually does. In `IdleCDOEpochVariant.requestWithdraw()`, wallet allowlisting is skipped when `keyringAllowWithdraw == true`. The upstream comment says this mode is "needed for liquidations", so it is a real operational state where withdraw requests go through even for wallets that are no longer Keyring-approved:

```
// IdleCDOEpochVariant.sol:145-151
if (!keyringAllowWithdraw && !isWalletAllowed(msg.sender)) {
    revert("not allowed");
}
```

So if the Pareto vault turns on `keyringAllowWithdraw` but the fund wallet is no longer Keyring-approved, the upstream vault would accept a withdraw request from the fund, but `ParetoFund` rejects it locally at `create()`. Wrapped AA holders then cannot exit through Grunt even though the underlying vault is letting withdrawals through.

Recommendation: There are two ways to handle this:

- **Option 1 - Document and accept:** Document that Grunt does not support Pareto's open-withdraw mode (`keyringAllowWithdraw`). If the Pareto vault enables it, Grunt redeem orders still require the fund wallet to be Keyring-approved.
- **Option 2 - Code fix:** For redeem orders, mirror the upstream rule: allow if `isWalletAllowed(address(this))` is true, or if the vault has `keyringAllowWithdraw == true`. Only gate deposit orders on `isWalletAllowed` alone.

3F: Fixed in PR 181.

Cantina Managed: Fix verified.

3.3.23 First deposit into a fresh Pareto tranche gets stuck because of `MIN_LIQUIDITY` burn

Severity: Low Risk

Context: *(No context files were provided by the reviewer)*

Description: `ParetoFund.create()` quotes the expected AA output for deposits from `virtualPrice()`. On a fresh Pareto tranche with zero supply, that quote is basically a full 1:1 mint.

But the upstream tranche token (`IdleCDOTranche`) burns `MIN_LIQUIDITY = 1000` raw tranche units on the very first mint:

```
// IdleCDOTranche.sol (upstream)
if (totalSupply() == 0) { _mint(address(1), MIN_LIQUIDITY); amount -= MIN_LIQUIDITY; }
```

`ParetoFund.commit()` records the post-mint AA balance delta as `depositReceived`:

```
// ParetoFund.sol:230
$.depositReceived = IERC20(_aaTranche).balanceOf(address(this)) - _before;
```

`_state()` then needs `depositReceived >= order.output` before the order can unlock:

```
// ParetoFund.sol:402-403
uint256 _expectedOutput = $.hasResolvedAmounts ? $.resolvedOutput : order.output;
uint256 _received = $.depositReceived;
```

So on the first-ever deposit into a fresh Pareto AA tranche, an order created with the exact quoted output gets stuck in `PROCESSING` because it is short by 1000 raw AA units, even though the deposit actually went through. The order cannot move forward through normal `unlock()` and needs an operator `resolve()` call to unstick it.

The shortfall is tiny and only hits the first deposit on a fresh tranche, but the order gets stuck for real.

Recommendation: There are two ways to handle this:

- **Option 1 - Document and accept:** Document that the first exact-quote deposit into a fresh Pareto tranche will need an operator `resolve()` call because of the upstream `MIN_LIQUIDITY` burn.
- **Option 2 - Code fix:** On fresh tranches (`aaTranche.totalSupply() == 0`), subtract the `MIN_LIQUIDITY` burn from the expected output before using it as the unlock threshold. Or handle the first deposit as a special case in `create()` or `commit()`.

3F: Acknowledged.

Cantina Managed: Acknowledged.

3.3.24 `TransferGuard.canTransfer()` is skipped entirely when a deposit/withdrawal produces zero share delta

Severity: Low Risk

Context: *(No context files were provided by the reviewer)*

Description: The transfer guard only runs through `_beforeTokenTransfer()`, which only fires when PM shares are actually minted, burned, or transferred. `TransferGuard.canTransfer()` checks blocklist status, whitelist mode, native-only mode, and `checkCollateralAllowed`, but none of that runs if there is no share movement.

`withdraw()` settles compliance at the end via `_settleShares()`. If the operation ends with a zero share delta (total assets unchanged), `_settleShares()` does not call `_mint()` or `_burn()`, so `_beforeTokenTransfer()` never runs:

```
// PositionManagerLP.sol:212-218
} else {
    // Assets unchanged: no mint/burn needed, but check pause since _beforeTokenTransfer
    ↪  won't run
    address guard = _storage.transferGuard;
    if (guard != address(0) && ITransferGuard(guard).paused(address(this))) {
        revert CommonErrors.Paused();
    }
}
```

The zero-delta branch only checks `paused()`. Blocklist, whitelist, native-only, and `checkCollateralAllowed` are all skipped.

The same gap exists in `deposit()` when the share mint rounds to zero: `_settleShares()` hits the `sharesToMint == 0` branch (line 198-202) and only checks `paused()`.

Right now the only callers of `deposit()` / `withdraw()` are the Facility (`MINTER_ROLE`), which would pass all guard checks anyway. But if a future minter is added that is blocklisted or not collateral-allowed, they could do a neutral withdrawal (repay debt and withdraw matching collateral, zero share delta) and the guard would never see them.

Recommendation: In the zero-delta and zero-mint branches of `_settleShares()`, call `canTransfer()` against `msg.sender` instead of only checking `paused()`. This keeps all guard modes (blocklist, whitelist, native-only, collateral-allowed) enforced even when no shares move.

3F: Fixed in PR 176.

Cantina Managed: Fix verified.

3.3.25 Executor can run facilitator-level operations on any intent's pooled LP capital

Severity: Low Risk

Context: *(No context files were provided by the reviewer)*

Description: `MorphoFlashLoanRequest` has `FACILITATOR_ROLE` on the Facility. Anyone with `EXECUTOR_ROLE` can call `execute()` to run whitelisted scripts that do facilitator-level things on any intent:

```
function execute(
    uint256 flashLoanAmount,
    SetRequestParams calldata requestParams,
    address script,
    bytes calldata scriptPayload
) external onlyRoles(EXECUTOR_ROLE) nonReentrant {
```

The executor picks which script to run, what parameters to use (how much to deposit, how much to borrow, when to do it), and which intent to target. The scripts (`SyncDeposit` , `SyncWithdrawal`) don't do any per-intent auth checks. They just take an `intentId` and run the full flow: `create()` → `commit()` → `unlock()` → `depositManager()` / `withdrawManager()` .

Guardian signatures for `setRequest()` control which intent the executor can bind to, but they don't control what the executor does after binding. Once bound, the executor decides everything.

LPs deposit into an intent through `FacilityLP.deposit()` , the facility moves that pool through fund/PM operations, and LPs get their share back through `FacilityLP.claim()` . The executor can make moves on that pool that should be the facilitator's call.

Maybe `EXECUTOR_ROLE` is supposed to be a lower-trust role (a bot or keeper), not the same as `FACILITATOR_ROLE` which controls how LP money gets moved. But through `MorphoFlashLoanRequest`, the executor gets the same power as a facilitator over pooled capital.

Recommendation: Document that `EXECUTOR_ROLE` on `MorphoFlashLoanRequest` is the same trust level as `FACILITATOR_ROLE`, since it can make decisions on any intent's LP capital. If the executor is supposed to be lower-trust, add per-intent parameter limits (e.g., allowed deposit ranges, timing windows) that the facilitator sets up front and the executor can only work within.

3F: Acknowledged.

Cantina Managed: Acknowledged.

3.3.26 Attacker can grief intent LP deposits by posting fake deposits up to `depositCap`

Severity: Low Risk

Context: FacilityLP.sol#L37-L43

Description: The facility allows LPs to deposit assets for an intent, up to a pooled total of `depositCap` configured at time of intent creation.

`FacilityLP.sol` logic also allows the LP to withdraw all of their deposits, if the current timestamp lies within the depositing phase. This flexibility to withdraw can be used by an attacker to fill the `depositCap` with fake deposits that add to the `totalSupply` and block new deposits temporarily.

- Attacker can deposit huge amounts of assets, fill the deposit cap.
- Then withdraw well within the deposit phase.
- The `resolveTimestamp` is reached, but now the intent's `totalSupply` is empty again.
- This will prevent genuine users from depositing their assets, and once the `resolveTimestamp` is reached no new deposits are possible.

They can even do this repetitively by sandwiching genuine deposits, inflating the `totalSupply` and making new deposits revert when the deposit cap is checked.

Recommendation: There are a few solutions that can be adopted to mitigate this griefing vector :

1. Use a `TransferGuard` with whitelist mode only, for the ERC-6909 LP tokens. This will help filter which addresses can deposit to that intent.
2. Document that the protocol team will ensure large deposit caps such that the cost of such an attack is high as the capital might need to be locked for a certain duration.
3. Document that the protocol team will preemptively lock intents when the `totalSupply` is near `depositCap` and such griefing is observed.

3F: Acknowledged. We will look at potential manipulations.

Cantina Managed: Acknowledged.

3.3.27 `resolve()` function emits incorrect `orderID` in `USCCFund`

Severity: Low Risk

Context: USCCFund.sol#L363-L366

Description: In `resolve()` function, the order struct is modified to update the input and output amounts as per the resolved amounts.

```
=>  order.input = input;
=>  order.output = output;

emit OrderResolved(_storage.currentOrderId, order.toId(address(this)), input, output,
 ↪  msg.sender);
```

Then the `OrderResolved` event is emitted with the orderID calculated from the order struct. Since orderID is calculated from hash of entire Order struct and we have changed the `order.input` and `order.output` values in this struct, it emits an entirely different orderID from what was actually resolved.

Recommendation: Consider using the orderID calculated from the original Order struct.

3F: Fixed in PR 142.

Cantina Managed: Fix verified.

3.3.28 Admin rights might get lost in `UpgradeableBeacon` contracts

Severity: Low Risk

Context: RequestFactory.sol#L81-L85

Description: The `RequestFactory` contract is used to deploy new proxies for Request, PT token and YT token contracts, all pointing to their respective beacon contracts to fetch the current implementation address.

The `REQUEST_BEACON`, `PT_TOKEN_BEACON` and `YT_TOKEN_BEACON` contracts are all set up as UpgradeableBeacon contracts (from solady). These inherit from solady's OwnableRoles library.

Here is the constructor logic from `RequestFactory`:

```
constructor(address intialBeaconOwner) {
  REQUEST_BEACON = address(new UpgradeableBeacon(intialBeaconOwner, address(new
   ↪  Request())));
  PT_TOKEN_BEACON = address(new UpgradeableBeacon(intialBeaconOwner, address(new
   ↪  Vault(false))));
  YT_TOKEN_BEACON = address(new UpgradeableBeacon(intialBeaconOwner, address(new
   ↪  Vault(true))));
}
```

There is no check for `initialBeaconOwner` here. If `address(0)` is passed, the owner of all beacons will be `address(0)`, which means nothing can be upgraded from those beacons; there are no upstream checks for owner address in solady as well.

Even if the owner is initialized to a valid address on these beacons, there is a `renounceOwnership()` function on these beacon contracts. If this function is called by the current owner by mistake, owner rights will be lost forever.

The `UpgradeableBeacon` contracts themselves are not upgradeable, so there is no way to rectify this.

Recommendation: Consider overriding the `renounceOwnership()` function to always revert, and add a zero address check for `initialBeaconOwner` in `RequestFactory` constructor logic.

3F: Acknowledged.

Cantina Managed: Acknowledged.

3.4 Informational

3.4.1 Small fixes and documentation Issues

Severity: Informational

Context: *(No context files were provided by the reviewer)*

Description: Various small issues, comments, and documentation gaps grouped into one finding:

- Facility.sol#L180 The `guardKey` name is misleading. It reads like an abstract key, but it is used as an address reference, not just a generic key.
- The README line 252 explicitly says `updateTarget` requires DEPOSITING state, but the code uses `getIntent(id)`, which has no state check and works in any state including RESOLVING.
- RequestFactory.sol#L80-L84 `intialBeaconOwner` => `initialBeaconOwner`.
- LibPause.sol#L45 `casting to 'uint40' is safe because pauseUntil is equal or less than PERMANENT_PAUSE` => `safe because the value is capped to PERMANENT_PAUSE before casting.`.
- WrappedAsset.sol#L128 If you set the `to` in burn to USCC address, then it's considered as a redemption. While modifying the wrapping token for USCC is not the right approach, this should be documented as a peculiarity.
- MorphoBorrowPosition.sol#L400 Include `seizedAssets` in the callback so liquidators can receive it as a parameter rather than doing a balance before/after check to determine how much collateral they received.
- WrappedAsset.sol#L167 The Solady ERC20 implementation differs from OpenZeppelin and allows transfers to address(0), which acts as a fake burn. Consider documenting this behavior difference from OpenZeppelin. EIP-20 does not restrict transfers to address(0).
- ParetoFund.sol#L110 | CentrifugeFund.sol#L105 | USCCFund.sol#L81 In Pareto and Centrifuge funds the wrapped-share variable is called `wrappedShare` and there is a check that `wrappedShare.underlying == share token`. In USCC fund the wrapped variant is called WUSCC with no equivalent check.
- VaultController.sol#L76 `SafeTransferLib.balanceOf` should be replaced with `IERC20.balanceOf`.
- PositionManagerRebalancing.sol#L104-L111 Morpho's `repay()` uses `toSharesDown` while `borrow()` uses `toSharesUp`. A value-neutral repay+borrow creates 1-2 wei phantom NAV loss in `PositionManagerRebalancing.sol.rebalance()`. With `maxRebalanceLoss=0` (set only when params are nonzero in `initialize()`), the loss check reverts for any loss > 0, so `maxRebalanceLoss == 0` should never be used.
- IFund.sol#L65-L69 `commit()` NatSpec says it transfers assets and returns the assets committed, but in `REDEEM` mode the committed input is shares, not assets.
- IFund.sol#L72-L76 `recover()` NatSpec says it recovers input assets and returns the assets recovered, but in `REDEEM` mode the recovered input is shares.
- IFund.sol#L79-L85 `unlock()` NatSpec returns the unlocked amount generically as assets, but in `DEPOSIT` mode the unlocked output is shares. Only `REDEEM` unlocks assets.
- IFacilityFunds.sol#L35 `amount` is documented generically as "The amount to include in the order," even though its meaning changes by mode: asset input for `DEPOSIT`, share input for `REDEEM`.
- IFacilityFunds.sol#L36 `minAmountOut` is documented generically as "The minimum amount expected from the order," even though its meaning changes by mode: share output for `DEPOSIT`, asset output for `REDEEM`.

- ParetoFund.sol#L344-L346 `totalAssets()` NatSpec says `virtualPrice` is in underlying-token decimals, but the implementation treats `virtualPrice` as WAD-scaled and divides by `1e18`.
- LibTokenController.sol The storage slots chosen in this library are the ones from ERC20 Solady but that is not explicitly stated, which can create confusion. The rest of the slots, like in Request, have an explanation.
- FacilityIntents.sol The `updateTarget()` function is able to change both the targetAsset and the guardKey. So it should be renamed accordingly. Same applies to the `updateTargetAsset()` function in LibIntents.sol.
- LibIntent.sol The Natspec says "Sets the deposit asset and quorum", but this function also sets transferableIntent boolean value.

Recommendation: Fix the issues listed above.

3F: Fixed in PR 188.

Cantina Managed: Fix verified.

3.4.2 `IFund.totalAssets()` reports wrapper-wide AUM, not per-fund holdings

Severity: Informational

Context: *(No context files were provided by the reviewer)*

Description: `USCCFund.totalAssets()`, `CentrifugeFund.totalAssets()`, and `ParetoFund.totalAssets()` all compute their return value from the global `totalSupply()` of the shared wrapper token, not from holdings scoped to the queried fund instance:

```
// USCCFund.sol:383-386
function totalAssets() external view override returns (uint256) {
    return WUSCC.totalSupply().mulDiv(_getOraclePrice(), _SCALED_UNIT);
}

// CentrifugeFund.sol:428-431
function totalAssets() external view override returns (uint256) {
    return
    ↪  ICentrifugeVault($.vault).convertToAssets(IERC20($.wrappedShare).totalSupply());
}

// ParetoFund.sol:344-347
function totalAssets() external view override returns (uint256) {
    return IERC20($.wrappedShare).totalSupply().mulDiv(IIdleCDOEpochVariant($.vault).vir
    ↪  tualPrice($.aaTranche),
    ↪  1e18);
}
```

When multiple fund instances share the same wrapper, calling `totalAssets()` on any one of them returns the combined AUM of all funds sharing that wrapper. Two funds each holding 1M USDC worth of USCC will each report 2M.

The `IFund` interface does not document this, so a caller querying `totalAssets()` on a single fund instance reasonably expects to receive the AUM of that fund.

Recommendation: Document in `IFund` and in each fund's NatSpec that `totalAssets()` may reflect wrapper-wide AUM when multiple funds share the same wrapper. Consider renaming the function to avoid confusion.

3F: Fixed in PR 186.

Cantina Managed: Fix verified.

3.4.3 Factory registries have no way to retire entries, so old funds, requests, and PMs stay permanently valid

Severity: Informational

Context: *(No context files were provided by the reviewer)*

Description: All three factories maintain an append-only boolean registry that records deployed components. Once an address is registered it can never be removed or marked inactive:

```
// USCCFundFactory.sol:76, 132, 140
mapping(address => bool) internal _isFund;
// createFund(): _isFund[fund] = true;
function isFund(address fund) external view returns (bool) { return _isFund[fund]; }

// PositionManagerFactory.sol:69, 114, 124
mapping(address => bool) internal _isPositionManager;
// create(): _isPositionManager[positionManager] = true;
function isPositionManager(address pm) external view returns (bool) { return
↪  _isPositionManager[pm]; }

// RequestFactory.sol:68, 131, 139
mapping(address => bool) internal _isRequest;
// create(): _isRequest[request] = true;
function isRequest(address request) external view returns (bool) { return
↪  _isRequest[request]; }
```

These registries work as on-chain deployment logs, not as live/valid indicators. There is no `removeFund()`, `discontinueFund()`, or equivalent on any factory.

There is no on-chain way to flag a fund as deprecated, since `isFund()` keeps returning `true` for all old fund addresses. A facilitator that queries `isFund()` to validate a fund before wiring it to an intent gets a positive result for a retired fund.

Recommendation: There are two ways to handle this:

- **Option 1 - Document and accept:** Leave the factories as deployment logs and handle discontinuation off-chain through operational monitoring of which funds, PMs, and requests are still active.
- **Option 2 - Code fix:** Make the on-chain registry a three-state enum instead of a boolean:

```
enum Status { NONE, ACTIVE, DISCONTINUED }
mapping(address => Status) internal _status;
```

Add an owner-callable `discontinue(address)` function to each factory that moves the entry from `ACTIVE` to `DISCONTINUED`, and update `isFund()` / `isPositionManager()` / `isRequest()` to return the `Status` value rather than a plain bool. Callers can then tell the difference between an address that was never registered (`NONE`), one that is currently valid (`ACTIVE`), and one that has been explicitly retired (`DISCONTINUED`).

3F: Acknowledged.

Cantina Managed: Acknowledged.

3.4.4 Rotating the TransferGuard silently clears the blocklist for all LP holders across every intent on that PM

Severity: Informational

Context: *(No context files were provided by the reviewer)*

Description: `PositionManagerAdmin.setTransferGuard()` swaps the PM's guard pointer to a new contract:

```
function setTransferGuard(address transferGuard_) external override onlyOwner {
    LibStorage.positionManagerStorage().transferGuard = transferGuard_;
    emit IPositionManagerAdmin.TransferGuardSet(transferGuard_);
}
```

The new guard starts with a clean state. It doesn't know about any addresses that were blocklisted or whitelisted in the old one, and nothing carries over.

`Facility._beforeTokenTransfer()` reads the guard dynamically on every mint, transfer, and burn:

```
(, address guard) = IPositionManager(_intent.properties.guardKey).config();
if (guard != address(0)) {
    if (!ITransferGuard(guard).canTransfer(_intent.properties.guardKey, from, to,
    ↪  amount)) {
        revert LibFacilityErrors.TransferBlocked(guard, from, to, amount);
    }
}
```

The moment `setTransferGuard()` is called, the new guard is live for all intents keyed to that PM. Any address that was blocklisted in the old guard is silently unblocked and can now transfer or burn LP shares freely. Nothing warns you or forces you to re-populate the new guard's blocklist before the swap goes live.

A PM owner who rotates the guard for any routine operational reason (contract upgrade, key rotation, compliance tooling change) must manually re-add every address status in the new contract first, or all previously restricted addresses become unrestricted the instant the new guard goes live.

Recommendation: There are two ways to handle this:

- **Option 1 - Document and accept:** Document that guard rotation clears the blocklist, and require an off-chain pre-migration checklist before rotating guards.
- **Option 2 - Code fix:** Before setting a new guard, require that all existing blocked addresses have been migrated. Pre-populate the new guard with all current blocklist entries before calling `setTransferGuard()`, and verify the new guard's state off-chain before the swap. Alternatively, expose a view on the old guard that lists blocked addresses to make migration auditable.

3F: Acknowledged. We just need to be very careful when changing the transfer guard. And we not necessary want the same addresses, so we don't want to enforce this.

Cantina Managed: Acknowledged.

3.4.5 PT/YT tokens are not fully ERC-20 compliant

Severity: Informational

Context: *(No context files were provided by the reviewer)*

Description: PT and YT tokens present themselves as `IERC20` implementations and annotate the surface with `@inheritdoc IERC20`, but the controller-backed behavior does not fully match the ERC-20 spec. There are three concrete gaps:

1. Missing `Transfer` event on zero-value transfers.

ERC-20 requires: "Transfers of 0 values MUST be treated as normal transfers and fire the Transfer event." The PT/YT controller only emits `Transfer` when the amount is strictly positive.

```
// TokenController.sol:82-83
// Transfer event emitted only for amount > 0
// zero-amount transfers succeed silently with no event
```

`src/request/abstract/tokens/TokenController.sol`

2. Missing `Approval` event on idempotent approvals.

ERC-20 requires `Approval` on every successful `approve(...)` call. The controller only emits `Approval` when the stored allowance actually changes.

`src/request/abstract/tokens/TokenController.sol`

3. Large allowances are not exact.

Values at or above `2^128 - 1` are clamped to a `uint128` sentinel and later read back as `type(uint256).max`, so the on-chain allowance differs from what the caller approved.

```
// TokenController.sol:107-108 (clamp) and 220-223 (normalization)
// Stored allowance is uint128; values >= uint128.max stored as sentinel
// allowance() view normalizes that sentinel to type(uint256).max
```

`src/request/abstract/tokens/TokenController.sol`

PT and YT `transfer(...)`, `transferFrom(...)`, and `approve(...)` all route through this controller:

`src/request/abstract/tokens/ControlledToken.sol`

The local `IERC20` interface claims to be "Interface of the ERC20 standard as defined in the EIP":

`src/interfaces/integrations/IERC20.sol`

Recommendation:

- Emit `Transfer` for zero-value PT and YT transfers.
- Emit `Approval` on every successful `approve(...)`, even when the effective stored value is unchanged.

3F: Fixed in PR 178.

Cantina Managed: Fix verified.

3.4.6 Blocking the current fee recipient bricks PM fee accrual and fee rotation

Severity: Informational

Context: *(No context files were provided by the reviewer)*

Description: `PositionManagerBase._accrueFees()` mints newly accrued fee shares directly to `feeRecipient`. That mint goes through `PositionManager._beforeTokenTransfer()`, which checks the live `TransferGuard` policy. If the current fee recipient later gets blocked under that guard, fee accrual reverts with `TransferBlocked`.

In `PositionManagerBase.sol` lines 93-105:

```
function _accrueFees() internal returns (uint256 currentTotalAssets) {
    // ...
    if (feeShares > 0) {
        address feeRecipient = LibStorage.positionManagerStorage().feeData.feeRecipient;
        _mint(feeRecipient, feeShares);
        emit IPositionManagerLP.FeesAccrued(feeRecipient, feeShares);
    }
    // ...
}
```

The `_beforeTokenTransfer()` hook in `PositionManager.sol` lines 207-214 checks the live guard for every mint:

```
function _beforeTokenTransfer(address from, address to, uint256 amount) internal
↪   override {
    address guard = LibStorage.positionManagerStorage().transferGuard;
    if (guard != address(0)) {
        if (!ITransferGuard(guard).canTransfer(address(this), from, to, amount)) {
```

```
            revert LibManagerErrors.TransferBlocked();
        }
    }
}
```

`_accrueFees()` is called first in every major PM entrypoint. `deposit()`, `withdraw()`, and `burn()` all call it before their main logic (`PositionManagerLP.sol` lines 51, 95, 131). `addBorrowModule()`, `removeBorrowModule()`, and `rebalance()` also call it first (`PositionManagerAdmin.sol` lines 33, 61). `setFeeData()` at `PositionManagerAdmin.sol` line 158 also calls `_accrueFees()` first before rotating the recipient, so the normal fee-rotation recovery path is itself blocked.

Once fees are pending and the current recipient is blocked, the PM deadlocks: every state-changing entrypoint reverts at the accrual step, including the owner's `setFeeData()` call to switch to a different recipient. A malicious or compromised compliance actor can trigger this by blocking the current fee recipient after fees have accrued.

Recommendation: Rotate before blocking: call `setFeeData(newRecipient, ...)` while the old recipient is still allowed, then block the old address.

3F: Acknowledged.

Cantina Managed: Acknowledged.

3.4.7 `ParetoFund` false-detects instant withdrawal and rejects the first normal redeem cycle

Severity: Informational

Context: *(No context files were provided by the reviewer)*

Description: `ParetoFund.commit()` tries to detect Pareto's instant-withdraw path by snapshotting `lastWithdrawRequest(address(this))` before and after `requestWithdraw()` and reverting unless the value increases:

```
// src/funds/pareto/ParetoFund.sol:236-241
uint256 _lwrBefore = IIdleCreditVault($.strategy).lastWithdrawRequest(address(this));
IIdleCDOEpochVariant(_vault).requestWithdraw(order.input, _aaTranche);
// lastWithdrawRequest won't change and the order would be stuck in PROCESSING.
if (IIdleCreditVault($.strategy).lastWithdrawRequest(address(this)) <= _lwrBefore) {
    revert LibFundsErrors.InstantWithdrawDetected();
}
```

That assumption is wrong in the first normal lending cycle. In the real `IdleCreditVault`, `epochNumber` starts at `0`. A normal epoch-gated withdrawal request records `lastWithdrawRequest[user] = epochNumber`, so a legitimate first-cycle withdrawal sets `lastWithdrawRequest = 0`. The value does not increase from its initial default of `0`, and `ParetoFund.commit()` misclassifies the normal request as an instant withdrawal and reverts `InstantWithdrawDetected()`.

On fresh Pareto vault deployments, redeem orders in Grunt are impossible until epoch 1.

Recommendation: Special-case epoch zero: treat `epochNumber == 0` with `lastWithdrawRequest == 0` as a potentially valid first normal request rather than an instant-withdraw false positive.

3F: Acknowledged. We will make sure to never use the first epochs.

Cantina Managed: Acknowledged.

3.4.8 `MorphoBorrowPosition.preLiquidate()` is missing `nonReentrant`, exposing a callback-reentry surface

Severity: Informational

Context: *(No context files were provided by the reviewer)*

Description: `MorphoBorrowPosition.preLiquidate()` is permissionless and has no reentrancy guard. It calls `MORPHO.repay(...)`, which reduces debt and collateral, then calls back into `onMorphoRepay()`, which withdraws the seized collateral to the liquidator and then calls `IPreLiquidationCallback.onPreLiquidate(...)` on the liquidator before pulling the loan tokens back.

During that callback, Morpho has already booked the repayment and the liquidator holds the seized collateral, but the loan tokens have not flowed back yet. Because `preLiquidate()` does not hold a lock, a malicious liquidator can re-enter `preLiquidate()` on the same contract from inside the callback, stacking nested in-flight liquidations. The same window is also open for oracle manipulation using the seized collateral and for read-only reentrancy into any external integrator that reads `PositionManager.totalAssets()` while the repayment is in progress.

Direct state-changing cross-contract reentry into `PositionManager` or `Facility` is currently blocked only because `deposit` / `withdraw` / `burn` and `depositManager` / `withdrawManager` / `burnManager` require specific roles (`MINTER_ROLE` and `FACILITATOR_ROLE` respectively). That protection comes from other contracts, not from `MorphoBorrowPosition` itself, and breaks the moment a future integration gives callback-capable liquidators access to either role.

Recommendation: Callback reentry into `preLiquidate()` is economically equivalent to sequential liquidation calls, so adding `nonReentrant` here mainly removes liquidator composability without fixing a real state divergence. The safety against cross-contract reentry already holds via role requirements on `deposit` / `withdraw` / `burn`.

Document that nested `preLiquidate()` reentry from the callback is expected behavior and economically identical to sequential calls. If a future integration gives callback-capable liquidators a role that can mutate PM state, revisit the guard then.

3F: Fixed in PR 184.

Cantina Managed: Fix verified.

3.4.9 `ParetoFund.create()` accepts orders that will always revert at `commit()` because it skips upstream mode checks

Severity: Informational

Context: *(No context files were provided by the reviewer)*

Description: `ParetoFund.create()` only checks local order shape, current `internalState`, wallet allowlisting, and a `virtualPrice`-based output check. It does not check the live Pareto vault mode gates that control whether the action will actually go through.

Two upstream gates are missing from `create()`:

Deposit side: `isDepositDuringEpochDisabled` makes `depositDuringEpoch()` revert, but Grunt already accepted the deposit order. `commit()` calls `depositDuringEpoch()` when `isEpochRunning()` is true:

```
// ParetoFund.sol:225-230
if (IIdleCDOEpochVariant(_vault).isEpochRunning()) {
    IIdleCDOEpochVariant(_vault).depositDuringEpoch(order.input, _aaTranche);
```

But upstream `depositDuringEpoch()` reverts when `isDepositDuringEpochDisabled == true`.

Redeem side: `allowAAWithdrawRequest` makes `requestWithdraw()` revert, but Grunt already accepted the redeem order. Upstream `startEpoch()` flips `allowAAWithdrawRequest = false`, so this is a normal mode, not an edge case.

`create()` only checks wallet allowlisting:

```
// ParetoFund.sol:151-153
if (!IIdleCDOEpochVariant($.vault).isWalletAllowed(address(this))) {
```

```
    revert LibFundsErrors.NotAllowedByFund();
}
```

It does not check `allowAAWithdrawRequest`, `isDepositDuringEpochDisabled`, or `isEpochRunning`. So the fund returns `ACCEPTED`, but the same order is guaranteed to revert at `commit()`. The stale accepted order stays live until a facilitator cancels it. Until then, it blocks the next order and can also block a bound Facility intent from moving forward.

Recommendation: There are two ways to handle this:

- **Option 1 - Document and accept:** Document that `ACCEPTED` from `create()` does not mean the order can actually be committed right now. The facilitator should check upstream mode gates before committing and cancel stale orders when the vault mode has changed.
- **Option 2 - Code fix:** Check the live upstream mode gates at `create()` time: for deposits, reject when the vault is running an epoch and `isDepositDuringEpochDisabled == true`. For redeems, reject when `allowAAWithdrawRequest == false` for the AA tranche.

3F: Fixed in PR 179.

Cantina Managed: Fix verified.

3.4.10 `USCCFund.create()` accepts redeem orders that will revert at `commit()` when Superstate accounting is paused

Severity: Informational

Context: *(No context files were provided by the reviewer)*

Description: `USCCFund.create()` checks local order shape, fund state, Superstate allowlisting, and the oracle-based output floor. It does not check whether the Superstate token is in `accountingPause`.

That matters on the redeem path. `USCCFund.commit()` for redeems burns `wUSCC`, unwraps to raw `USCC`, and then calls `USCC.offchainRedeem(order.input)`. Superstate documents `accountingPause` as pausing mint and burn, and `offchainRedeem()` is a burn path.

`create()` only checks allowlisting:

```
// USCCFund.sol:176-179
if (!ISuperstateToken(USCC).isAllowed(address(this))) {
    revert LibFundsErrors.NotAllowedSuperstate();
}
```

The redeem commit path at `USCCFund.sol:239-243` calls `offchainRedeem()`:

```
IWrappedAsset(WUSCC).burn(msg.sender, address(this), order.input);
ISuperstateToken(USCC).offchainRedeem(order.input);
```

So a redeem order gets accepted while Superstate accounting is paused, then always reverts at `commit()`. The revert rolls back balances so nothing is lost, but the order stays in `ACCEPTED` state. Until someone cancels it, the stale order blocks the next one.

Recommendation: There are two ways to handle this:

- **Option 1 - Document and accept:** Document that an `ACCEPTED` redeem order does not mean Superstate will let it go through right now. Make sure there is a clean cancel path when commit fails due to an accounting pause.
- **Option 2 - Code fix:** Check `accountingPaused()` on the redeem path in `create()` before accepting the order.

3F: Fixed in PR 180.

Cantina Managed: Fix verified.

3.4.11 `checkCollateralAllowed` passes PM share amount to `isAllowed()`, not the actual collateral amount

Severity: Informational

Context: *(No context files were provided by the reviewer)*

Description: When `checkCollateralAllowed` is enabled, `TransferGuard.canTransfer()` forwards the ERC20 transfer `amount` into `collateral.isAllowed(account, amount)`:

```
// TransferGuard.sol:166-170
if (config.checkCollateralAllowed) {
    (address collateral,) = IPositionManager(token).assets();
    if (from != address(0) && !IWrappedAsset(collateral).isAllowed(from, amount)) return
    ↪  false;
    if (to != address(0) && !IWrappedAsset(collateral).isAllowed(to, amount)) return
    ↪  false;
}
```

On PM burns, `amount` is the number of PM shares being burned. But the actual collateral released is computed separately in `PositionManagerLP.burn()`:

```
// PositionManagerLP.sol:143
collateral = _totalCollateral.mulDiv(shares, _totalSupply + virtualShareOffset_);
```

The collateral released can be larger than the share amount (e.g., when the PM has gains and each share is worth more than 1 unit of collateral). So `isAllowed()` gets the share count, not the collateral count.

Right now this does not matter because the only `isAllowed()` override in the codebase is `SuperstateRestrictedWrappedAsset`, which ignores the `amount` parameter entirely and just checks `USCC.isAllowed(account)`. But if a future collateral wrapper uses an amount-sensitive `isAllowed()` policy (e.g., per-address caps), the guard would check against the wrong number.

Recommendation: Document that `checkCollateralAllowed` passes the PM share transfer amount to `isAllowed()`, not the collateral amount. Any future `isAllowed()` override that uses the amount parameter will need a different approach to get the actual collateral value.

3F: Acknowledged.

Cantina Managed: Acknowledged.

3.4.12 Inconsistent zero address checks for owner

Severity: Informational

Context: Facility.sol#L62

Description: The `initialize()` function in `Facility.sol` checks and ensures that the owner address is not zero.

But this check is applied inconsistently across the codebase. The following contracts do not have zero address checks in their initialization logic:

- `WrappedAsset.sol`.
- `TransferGuard.sol`.
- `Request.sol`.
- `PositionManager.sol`.
- `MorphoRebalancer.sol`.

This is an issue because these contracts inherit from solady's `OwnableRoles` library and the upstream function `_initializeOwner()` in solady does not revert if the ”owner” is being set to address(0).

Recommendation: Consider adding consistent checks to ensure that a contract is never initialized with zero address as the owner.

3F: Fixed in PR 187.

Cantina Managed: PR 187 is a partial fix: it adds zero-address checks for the owner in `Request` and `PositionManager`. The remaining three contracts (`WrappedAsset`, `TransferGuard`, and `MorphoRebalancer`) were not updated as they are not planned for upgrade.